

ETX: A DSL-Agnostic, Chiplet-Aware Compiler for Dynamic GPU Megakernels

Author: Kailun Chen · Status: v0.3 design (pre-implementation, phase-0 calibration measured) · Date: 2026-09-22 · Target hardware: AMD MI300X (gfx942) first; CDNA2/CDNA4, RDNA, NVIDIA Hopper/Blackwell via the machine model

0. Summary

- ETX compiles a graph of tile-level operators into one persistent GPU kernel while keeping shape dynamism, data-dependent control flow (MoE routing, speculative decoding) and cross-step dynamism as first-class inputs. It adopts the Event Tensor abstraction of the ETC paper (arXiv 2604.13327 v2, MLSys 2026) and adds the two things that paper does not have: an explicit machine model and a cost-driven choice of scheduling policy.
- Three hard decisions are enforced in code: hardware differences exist only as data (etx/machine/arch/*.yaml; a test fails if an architecture name appears in a pass); capabilities (correctness) and costs (performance) are modelled separately; and static / dynamic / hybrid scheduling is chosen per subgraph by a cost model that prints its reasons. The paper's own numbers motivate the third: the same primitives give +9% with static and -17% with dynamic scheduling on Qwen3-32B TP=4.
- The performance model has two lower bounds (bandwidth and critical path) and four levers. At small batch the critical-path term dominates: a hand-written MI300X MoE megakernel (fleet-mi300x) spends 31 us of a 133 us layer moving bytes and about 100 us in fixed per-phase latency. ETX turns the four levers that hand-written kernels use (fewer phases, cheaper signals, cross-barrier prefetch, fine-grained events) into compiler passes.
- The chiplet is a first-class concept: an exec-domain tree, a domain event scope between cluster and device, per-scope fence / poll / memory-type lowering rows, and a hybrid scheduler that pushes only inside a domain and uses statically pre-ordered queues across domains.
- Phase-0 calibration on a rented MI300X (ROCm 7.2.4) on 2026-09-22 measured: workgroup k lands on $XCD(k+6) \bmod 8$, not the documented round-robin; one-way arrive latency 642 ns same-XCD / 742 ns cross-XCD when polling through L2 after an L1 invalidate (vs 873 / 892 ns with agent-scope atomic loads); a plain load never observes the arrive; release / acquire sequences from the lowering table produced 0 stale payloads in 20,000 trials; a ticket-based ready queue sustains 76.6 M pops/s at 512 poppers where a CAS-based ring collapses to 0.41 M pops/s.
- What exists today: the L1-L5 pipeline in Python (IR, six verification checks, seven passes, lowering-table code generation, protocol simulator), 46 passing tests, and the phase-0 benchmark programs. On 2026-09-22 the first ETX-generated megakernel (the split-K row-sum example) ran end to end on the MI300X with 608 cooperative workers and matched the CPU fp32 reference (max abs error $4.8e-7$) under static, hybrid and dynamic scheduling: 0.101 / 0.115 / 0.118 ms per step at $n=64$ and 0.128 / 0.441 / 0.512 ms at $n=1024$. Two defects visible only on hardware (ticket-ring wrap-around, event scope under global scheduling) are now encoded as runtime and pass rules. MoE and GEMM + reduce-scatter tiles are still stubs.

1. Executive Summary

1.1 What ETX is

ETX (Event Tensor eXtended) is a compiler that sits above tile-level DSLs. It does not generate the instructions inside a tile. It cuts operators into tiles, connects tile-level dependencies, chooses a schedule, places tasks on the machine, generates the skeleton of a persistent kernel, and links in tile bodies compiled by whichever DSL produced them (Triton, TileLang, CuTe DSL, Composable Kernel, hand-written HIP or CUDA).

The abstraction is the Event Tensor from the ETC paper: dependencies between tile-level tasks are expressed as multidimensional arrays of integer counters, and the mapping from task coordinates to event coordinates is an einsum-style expression such as "ij->i". ETX keeps that abstraction unchanged and extends it with an event scope (`workgroup < cluster < domain < device < system`), a domain assignment, an initialisation kind, and an optional epoch stamp.

Two additions distinguish ETX from the paper's implementation:

1. An explicit machine model. A machine is a tree of execution domains (for MI300X: one device, eight XCDs of 38 active CUs, each with a private 4 MB L2), a visibility table that says which fence, poll and memory type each event scope needs, a capability table (booleans and enumerations that decide whether an optimisation is legal) and a cost table (numbers that decide whether it is worthwhile). Passes and code generators read this object; they never test the architecture name.
2. A cost-driven scheduling decision. For every task grid the compiler chooses static, dynamic or hybrid scheduling from workload variance and topology costs, and it records the reason. Hybrid scheduling, which the paper does not have, uses a dynamic ready queue inside each domain and statically pre-ordered queues plus event counters across domains, so no push ever crosses a domain or a device.

1.2 Why

Low-batch decoding issues hundreds to over a thousand kernels per step; the shortest finish in about 2 us and the launch gap is 5-10 us, so launch overhead dominates (ETC paper, figure 1). CUDA Graphs remove the gap but keep the kernel boundary, an implicit global barrier. A megakernel replaces the boundary with tile-level dependencies: a tile of operator B starts as soon as the tiles of A it depends on have finished.

The ETC paper shows that a compiler can do this, and also shows the risk: on its MoE layer dynamic scheduling wins 4% over static, and on the dense TP=4 model the same dynamic scheduler is 17% slower than not fusing. The policy is a function of workload and interconnect topology and must be chosen by a cost model. The paper's implementation also assumes one machine (homogeneous SMs, one coherent L2, multimed, NVLink); on an MI300X none of these hold, and a direct port would be silently wrong before it was slow.

1.3 Expected outcomes

The performance model in section 4 bounds the gain. For dense models of 30B parameters and more at batch 16 and above, the baseline (vLLM / SGLang with CUDA Graphs and torch.compile) already runs at 55-60% of the bandwidth bound and the realistic gain is 10% to 30%. For MoE and tensor-parallel decoding at small batch, baselines run at 10% to 25% of the bound; 1.5x to 3x is physically available, but only if the critical path is shortened (levers 1 to 3), not merely by placing the operators in one kernel. ETC reached 1.15x on Qwen3-32B at TP=1 and 1.48x over vLLM on 30B-A3B at batch 1; fleet-mi300x, hand-written, reached 3.60 ms/token on DeepSeek-V2-Lite at batch 1 against vLLM's 4.52 ms on the same

MI300X. All ETX-side numbers in this document are targets or estimates unless marked measured; paper-side numbers are citations.

1.4 Delivered now versus later

Delivered now: the full L1-L5 pipeline in Python (IR, six verification checks, machine models for five architectures, seven passes with printed reasons, lowering-table code generation, protocol simulator), three example graphs, phase-0 microbenchmarks with first MI300X measurements, 46 tests, and a first end-to-end run of the generated split-K megakernel on MI300X with a host launcher, reference check and watchdog (section 14). Not yet delivered: MoE and GEMM + reduce-scatter tile bodies (stubs), CUDA emission on hardware, a runnable Triton host-DSL kernel, the cross-device cost `t_dev_ns` and cost tables for architectures other than gfx942, the cross-step control ring, and the communication-fusion path without `multimem`.

2. Background and Prior Art

2.1 The ETC paper: abstraction

The paper (Jin, Hou, Wang, Lai et al., “Event Tensor: A Unified Abstraction for Compiling Dynamic Megakernel”, MLSys 2026, arXiv 2604.13327 v2, CMU and NVIDIA) contributes an IR-level abstraction, not a new scheduler. It lifts “who waits for whom” from semaphores scattered through hand-written kernels into a first-class tensor in the compiler. Three language constructs:

Construct	Role
Device function	A function launched over a multidimensional coordinate space as a grid of tile-level tasks. Each task runs on one SM and may use warp specialisation and tensor cores. The coordinate space is symbolic, e.g. $(n, 4)$.
Event Tensor	A multidimensional array whose elements are events: an initial <code>wait_count</code> , <code>notify()</code> (atomic decrement) and <code>wait()</code> (spin until zero). Under dynamic scheduling reaching zero can also trigger consumer tasks. Shapes may contain symbolic dimensions.
Graph function	A computation graph of <code>call_device</code> nodes over data tensors and event tensors. Each call declares <code>in_edges</code> / <code>out_edges</code> with an einsum-style map (e.g. <code>"ij->i"</code>) from task coordinates to event coordinates.

The paper’s minimal example (figure 3) is split-K summation: `partial_sum` has `tile_num = (n, 4)` and notifies `E = ETensor((n,), wait=4)` through `"ij->i"`; `final_sum` has `tile_num = (n,)` and waits on `E[i]` through `"i->i"`. Row `i` of the second phase starts as soon as its four partial sums exist; with per-kernel execution it would wait for the whole first phase. One tensor represents thousands of events and the runtime never materialises a task graph.

Two kinds of dynamism enter the IR:

- Shape dynamism. Tensor and ETensor dimensions may be symbolic (batch, sequence length). The compiled artefact is a template instantiated with concrete values at run time; no recompilation and no graph re-capture. This is why the system can be compiled ahead of time.
- Data-dependent dynamism, through two mechanisms. Data-dependent update: which event a task notifies is decided by a runtime tensor, e.g. `"i->topk[i,:]"` (token `i` notifies the `k` expert events it is routed to), and the events’ initial counts are computed in the same tile that computes top-k. Data-dependent trigger: how many consumers an event triggers is decided at run time, e.g. `"i->range(indptr[i], indptr[i+1])"` (expert `i` triggers the number of GroupGEMM tiles it needs).

The invariant is that the dependency chain stays feed-forward (Attention, TopK, grouping, GroupGEMM); dynamism appears only in notification targets and trigger ranges, never as cycles or arbitrary control flow.

2.2 The two scheduling transforms

Dimension	Static scheduling (paper section 3.1)	Dynamic scheduling (paper section 3.2)
Who orders tasks	The host pre-orders a queue per SM before launch (round-robin in the paper) and materialises it in global memory	An on-chip scheduler: when an event reaches zero its consumers are atomically pushed to a ready queue; idle SMs atomically pop
How dependencies are enforced	<code>wait()</code> at task start, <code>notify()</code> at task end	The same notify / wait, plus push / pop
Shape dynamism	Queues are pre-ordered for a sampled set of shapes; an unseen shape reuses the queue of the next larger sampled shape	Native; order is decided at run time
Data-dependent dynamism	Conservative fallback: the affected notify / wait are rewritten to <code>E[0]</code> , which is a global barrier	Native
Overhead	Lowest: atomics and spinning only	Queue push / pop; the authors acknowledge contention on the centralised global-memory queue; appendix E moves the push off the critical path with early push (consumers are pushed when the producer is dispatched)
Suited to	Predictable workloads, latency-sensitive small batch, multi-GPU	Irregular workloads (MoE routing), communication jitter

After lowering, both leave two pieces of runtime state, one integer event tensor and the scheduler’s task queues, with no task-graph walker and no interpreter. The pipeline is: graph-level optimisation, tile-level optimisation, static or dynamic scheduling transform, prefetch rewriting, persistent-kernel code generation, static-queue materialisation.

2.3 Results

All numbers are from the paper, 8x B200 with NVLink.

Experiment	Baselines	Result	Schedule used
GEMM + Reduce-Scatter (TP=8, 8192 tokens)	cuBLAS+NCCL, TP-Async, Triton-Distributed, cuBLASmp	up to 1.40x (vs unfused)	dynamic (absorbs network jitter)
All-Gather + GEMM	same	up to 1.40x	static (ring order is predictable)
MoE layer, Qwen3-30B-A3B (128 experts, top-8)	Triton 3.4, FlashInfer 0.2.14	up to 1.23x (1024 tokens)	dynamic
End-to-end TPOT, 30B-A3B, bs=1	vLLM 0.11, SGLang 0.5.3 (CUDA Graph + torch.compile)	1.48x / 1.20x	static

Experiment	Baselines	Result	Schedule used
End-to-end TPOT, Qwen3-32B, TP=1	same	up to 1.15x / 1.09x	static
End-to-end TPOT, Qwen3-32B, TP=4	same	0.99-1.06x; SGLang is faster (lighter CPU-side scheduling)	static
Qwen3-32B engine warm-up	SGLang 583 s / vLLM 123 s (JIT + 51 / 67 graph captures)	35 s, 0 captures; offline compile 107 s	-
MoE layer, static vs dynamic (vs unfused megakernel of the same code)	1 / 128 / 1024 / 4096 tokens	static 1.03 / 1.02 / 1.04 / 1.02; dynamic 0.95 / 1.06 / 1.08 / 1.03	dynamic wins 4% on irregular load
Qwen3-32B TP=4, static vs dynamic	bs 1 / 16 / 32 / 128	static 1.09 / 1.06 / 1.07 / 1.06; dynamic 0.83 / 0.82 / 0.85 / 0.89	cross-device pushes make dynamic slower than unfused

The last two rows are the most important evidence for this design: with identical primitives, the wrong scheduling policy turns +9% into -17%.

2.4 Seven critical judgements

1. Events as integer counters must be kept. It compresses runtime state to two memory blocks and lets the symbolic-shape compiler infrastructure be reused directly. Any “event object” or “task-graph node” is a regression.
2. Scheduling is a compiler transform, not a runtime component. Static and dynamic are rewrites of the same IR; the runtime keeps only push and pop. This is what allows the policy to be chosen per subgraph rather than globally; ETX’s hybrid scheduling is built on it.
3. The machine model is implicit and holds only on B200. The paper assumes homogeneous SMs, a coherent unified L2, `multimem` reduction writes and NVLink. On MI300X at least four assumptions fail: L2 is eight private 4 MB segments; there is no `multimem`; cross-XCD atomics travel over Infinity Fabric; system-scope atomics have memory-type requirements. The paper’s claim that the abstraction is “not specific to a GPU generation” is about the IR; the implementation was not validated elsewhere.
4. Static scheduling’s fallback for dynamism is too coarse. Shapes reuse the next larger sampled queue; data dependence degrades to an `E[0]` global barrier. This is the worst case exactly for the most common combination (MoE, low batch, multi-GPU), which hybrid scheduling must cover.
5. The cost of dynamic scheduling is a function of topology. 0.83x at TP=4 shows that a push across devices never pays; the authors also acknowledge contention on the centralised queue. Queues must be hierarchical and pushes must have a domain boundary.
6. Tile quality and fusion gain must be decoupled. The authors acknowledge that their generated GEMM tiles are sometimes slower than cuBLAS, which explains the occasional TP=4 loss. The system must allow hand-written tiles (CUTLASS / CK style) to be inlined, and evaluations must report fusion gain separately from tile quality.
7. One launch per step, no loops inside the graph. The paper’s megakernel covers one decoding iteration; continuous batching is absorbed by symbolic shapes but the kernel restarts every step. Cross-step persistence (a resident kernel fed step descriptors through a ring buffer) is untouched and is another source of gain at low latency.
8. Only task-level dependency transforms, no graph-changing transforms. ETC can split an operator into tiles and refine dependencies, but it never removes a phase: no recomputation in exchange for an event, no cross-barrier weight prefetch. At small batch the critical path is set by the number of phases

and the fixed latency per phase, which is exactly where hand-written kernels (Hazy, fleet-mi300x) pull ahead. Section 4 quantifies this.

(The specification’s heading says seven; its list carries the eighth as the consequence developed in section 4.)

2.5 The 2026 landscape: ten design axes

Within a year of the paper more than a dozen megakernel systems appeared. Their differences concentrate on ten axes; every ETX choice has a precedent or a counter-example in this table.

Axis	Choices in existing systems	ETX choice and reason
Who owns the scheduler	Dedicated scheduler SMs (MPK: 4 SMs; Fleet: one workgroup per XCD) · every SM pulls from a global counter (Hazy 70B, Cohere) · host pre-orders, no in-kernel scheduler (Hazy 1B, Ada-MK, Kog, ForgeMegakernel; MPK’s 2026 roadmap also moves to fully static)	Chosen per subgraph: static segments have no scheduler; dynamic segments have one local queue per domain and workers pull. No dedicated scheduler workgroups (in fleet-mi300x such a workgroup only mirrored events; 2.6% of CUs is irrelevant when bandwidth-bound and a loss when compute-bound).
Queue topology	Per-worker queues (MPK’s JIT/AOT pair, Fleet) · one global queue (Hazy 70B) · per-operator drain queues (Cohere’s ATTN_DRAIN / MOE_*_DRAIN) · no queue (Kog, Ada-MK)	Domain-local queue plus a static table per worker; data-dependent operators may add per-operator drain queues (Cohere’s split is orthogonal to the domain split and composes with it).
When dependencies are resolved	JIT (dispatch after the event fires) vs AOT (pre-enqueued, worker spins): MPK mixes both, Cohere and Hazy are fully AOT, Ada-MK resolves at compile time	Static segments AOT spin (one hop); dynamic segments JIT (two hops); ETC’s early push is the default in dynamic segments.
Event granularity and encoding	Event objects with fan-in / fan-out ranges (MPK) · one counter per edge (Hazy, Forge) · one target count per task (Cohere) · sentinel values inside the data buffer (Kog: 0.8 us vs 7.6 us for atomics) · flags with epochs (mKernel)	Integer event tensor plus epoch stamp: a resident kernel need not clear counters between steps, <code>wait</code> compares <code>count == epoch_target</code> ; sentinel values are an optional lowering (capability bit <code>sentinel_signal</code>).
Memory model of signals	GPU-scope atomics plus threadfence (NVIDIA systems) · chiplet hierarchy: L2-local counts plus one <code>sc0</code> <code>sc1</code> fence per XCD (Fleet, 14.5x fewer cross-chiplet signals) · compiler-visible acquire / release tokens (Triton-Distributed <code>consume_token</code> , Iris block / gpu / sys scopes)	Scope is in the IR; the lowering table gives the fence per architecture; last-arriver writes back once (validated by Fleet and fleet-mi300x).
Dynamic shape	Specialised graphs per batch size (MPK, Fleet, ETC-static) · fixed at compile time (Forge, Hazy 1B, Ada-MK) · work stealing only for data-dependent operators (Cohere, MPK’s JIT attention) · role adaptation (mKernel) · deterministic padding (MoK)	Symbolic-shape template plus shape buckets; local queues for data-dependent segments; no padding fallback.
Shared memory / LDS ownership	Page allocator with instruction-tracked lifetimes (Hazy 13 x 16 KB pages, Forge, Ada-MK’s page state machine) · worst-case allocation per task type (MPK, Fleet; the source of occupancy complaints)	The L5 skeleton has a built-in LDS page allocator so the next tile’s weight load overlaps the current tile’s write-back; page count derived from resource class and LDS capacity (64 / 160 / 228 KB).

Axis	Choices in existing systems	ETX choice and reason
Code-generation path	Superoptimiser plus transpiler (MPK) · MLIR (Ada-MK, cuTile) · hand-written plus tile library (Hazy, Cohere, MoK, Kog) · agent synthesis plus checker (Forge, AutoMegaKernel) · Triton plus SHMEM primitives (Triton-Distributed, Iris, DITRON)	No side taken: link mode accepts hand-written and vendor-library tiles, host-DSL mode accepts Triton; ETX generates the scheduling skeleton.
Hardware coverage	NVIDIA almost exclusively; AMD only Fleet (MI350X, MPK derivative), Kog (MI300X, bespoke), Iris / DITRON (communication primitives); Rubin claims hardware tile-level dependency triggering	Chiplet is first-class; wait may lower to a hardware trigger (capability bit <code>hw_tile_trigger</code>), in which case ETX simply omits the spin loop.
Fusion scope	Whole decode step (MPK, Hazy, Cohere, Forge, Kog) · one layer type (FlashInfer MegaMoE, MegaGDN, MoK) · the serving loop itself (Blink, MPK's in-kernel admission)	Subgraph level: parts that fail to fuse fall back to per-kernel execution; cross-step residency is an optional mode.

Two data points deserve separate mention. Cohere (2026-09) reached 1.58x over vLLM on 30B-A3B at batch 1 on H100 with host-pre-ordered static waves plus per-operator drain queues, which shows hybrid is the right direction. AMD's Fleet on MI350X, using XCD-local L2 counts plus one GPU-scope fence per XCD, is 1.13-1.30x faster than a direct MPK port, which shows chiplet awareness is not optional.

2.6 Experience from fleet-mi300x

A hand-written megakernel of the same kind was built on MI300X (DeepSeek-V2-Lite, batch-1 decode, 3.60 ms/token, 20% faster than vLLM on the same machine; github.com/cklxx/fleet-mi300x, docs/STATUS.md). It established three facts that entered L3 and L5 directly: cross-XCD events cost an order of magnitude more than intra-XCD events, and one MoE layer could be reduced from 10 global events to 3 with 24 XCD-local events; the producer's `buffer_wb12` write-back and the consumer's L1 invalidate cannot be omitted; and the workgroup-to-XCD mapping of a persistent kernel is not a contract and must be discovered at launch.

3. Goals, Non-Goals and Design Principles

3.1 Goals, by priority

1. DSL-agnostic: Triton, TileLang, CuTe DSL, CK and hand-written HIP / CUDA all connect.
2. Hardware-portable: one IR produces correct and reasonable code on CDNA2 / CDNA3 / CDNA4, RDNA and NVIDIA Hopper / Blackwell; the chiplet is a first-class concept.
3. First-class dynamic workloads: shape dynamism, data-dependent dynamism, cross-step dynamism (continuous batching, speculative decoding).
4. Automated and explainable scheduling: the compiler chooses static / dynamic / hybrid from cost tables and prints the reason.
5. Verifiable: every fused kernel has an unfused reference for element-wise comparison, and a deadlock can be located.

3.2 Non-goals

- No training framework; the first version fuses inference only.

- No in-house generation of all tile-level code; tuned hand-written tiles may be inlined.
- No multi-node in the first version; the IR does not hard-code a single machine, but the scheduler sees only single-machine topology.
- No attempt to fuse every operator; failed fusion degrades gracefully to partial-subgraph fusion plus per-kernel execution.
- No arbitrary control flow; dependencies must be expressible as affine maps or runtime range / gather maps.

3.3 Three hard decisions

Decision	Content	Cost of violating it	Enforcement in code
Hardware differences exist only as data	No if arch == gfx942 in any pass. All differences live in the capability, cost and lowering tables of arch/*.yaml.	New hardware requires pass changes; every pass becomes a forest of conditionals	tests/test_no_arch_branches.py fails if an architecture name appears under ir/, passes/, sim/, codegen/ or frontends/
Capability and cost are modelled separately	A capability is a boolean or enumeration deciding whether something can be done; wrong means wrong code. A cost is a number deciding whether it is worth doing; wrong means slow.	It becomes impossible to answer “is this optimisation unsupported here, or merely unprofitable?”	capabilities: / visibility: blocks versus costs: blocks in each YAML; MachineModel.vis() raises on a missing scope; t_sync_us() falls back to a pessimistic value and logs a note when a cost is uncalibrated
Scheduling policy is chosen by the cost model	Each subgraph is decided static / dynamic / hybrid from workload features plus topology costs, with a reason attached.	A repeat of the paper’s 0.83x at TP=4	passes/p4_schedule_mode.py writes plan.reasons[grid] for every grid; tests/test_passes.py::test_reasons_are_printed

One discipline runs through every layer: dependency semantics, machine facts and placement decisions are never mixed. A wrong dependency gives wrong results or deadlock and lives in L2; a wrong machine fact gives silent errors or poor performance and lives in L3; a wrong decision is only slow and lives in L4. Each kind of failure is diagnosed in exactly one layer.

4. Performance Model

4.1 Two lower bounds

Before writing any pass, the accounting must show where a megakernel can take time from. The time of one decode step has two lower bounds; the larger applies:

```

T_step >= max( sum(bytes) / BW_HBM , # bandwidth term
               sum over phases ( bytes_phase / BW_HBM + T_fixed(phase) ) ) # critical-path
term

T_fixed(phase) = signal latency + drain (slowest worker minus the mean) + prologue ramp of
the next phase

```

At large batch the bandwidth term dominates and fusion recovers the holes at phase edges (wave quantisation, launch gaps): 5% to 15%. At small batch the critical-path term dominates and a megakernel is not “overlapping” but “shortening the chain”. The same technique gives multipliers an order of magnitude apart in the two regimes, and the reason is entirely how far the baseline is from its bound:

Scenario	Bytes per token	Bandwidth bound	Baseline	Megakernel	Reading
Llama-1B, H100 (Hazy)	about 2.4 GB	about 0.7 ms	vLLM about 2.5 ms (28% MBU)	under 1 ms (78% MBU)	Tiny model; baseline is drowned by hundreds of launches; hence 2.5x
Qwen3-32B, B200 (ETC)	about 66 GB	about 8.2 ms	vLLM about 14 ms (57%)	about 12.5 ms (65%)	Baseline already at 60% of bound; theoretical maximum 1.77x, achieved 1.15x
DeepSeek-V2-Lite MoE, MI300X (fleet-mi300x)	4.9 GB	0.93 ms	vLLM 4.52 ms (21%)	3.60 ms (26%)	The gap is not launches; it is 27 layers x 7 phases of serial fixed latency

4.2 Where one MoE layer’s 133 us goes (fleet-mi300x, measured total, estimated split)

Segment	Time	Nature
Bandwidth term (166 MB / 5.3 TB/s)	31 us	Moves bytes
Prologue data movement / folding	about 36 us	Critical-path term; measured
Signalling plus drain	about 20 us	Critical-path term; estimated
Serial small tasks with only a few workers active	about 46 us	Critical-path term; estimated

The layer has 7 phases x 3 global events plus 24 XCD-local events. Only the total and the prologue are measured; the split is an estimate, and phase-0 calibration is to replace it with a per-phase trace. The 26% MBU is not because tiles are slow: the GEMV in isolation reaches 93% of achievable bandwidth. It is because the kernel runs for about 100 us per layer while moving almost no bytes. The last three segments correspond to levers 2, 2 and 1 below; lever 3 fills them with the next phase’s bytes.

4.3 Four levers

Lever	What it removes	Method	Pass	Floor
1. Shorter chain	Unnecessary phase boundaries	Recomputation in exchange for events: small producers are inlined into consumers’ prologues; the event and the task are deleted (Pass 3)	L4 Pass 3	Only the natural all-to-all dependency points remain: 3 in a MoE layer (o_proj needs all heads, the router needs the full vector, the next layer needs the sum over all experts)

Lever	What it removes	Method	Pass	Floor
2. Cheaper links	Signal latency, prologue	XCD-local events; sentinel values instead of counters (0.8 vs 7.6 us); last-arriver write-back; prologue data movement removed in tile design (once per XCD rather than once per worker)	L5 lowering table; L1 tile contract	Signal about 0.2 to 0.8 us per link
3. Cross-barrier overlap	Idle spinning while waiting on events	Weights depend on no event: while spinning, a worker streams its share of the next phase's weights into LDS / registers and computes immediately when the event fires (Hazy's main route to 78% MBU)	L4 Pass 7 + LDS paging + resource classes	Fixed latency is covered by the next phase's bytes
4. Fine-grained events	Drain (waiting for the slowest worker)	Global events split per head / per row (the paper's $ij \rightarrow i$); consumers wait only for their producers; a straggler delays only its own downstream	L2 edge maps + Pass 2 partitioning	True all-to-all points still wait for everyone

Together, a layer moves from “7 x (bytes / bandwidth + fixed)” to “3 x bytes / bandwidth + 3 x fixed latency covered by prefetch”, which is the only way to approach the 31 us bandwidth term. Of the four levers ETC has only the fourth; MPK and Fleet have part of the second; Hazy hand-wrote the first three. ETX implements all four as passes; that is its route to catch up with hand-written kernels and its main increment over ETC.

4.4 Cost model coefficients

```

T_sync(edge) = T_local    if producer and consumer share a domain
               = T_cross   if they cross a domain (through the shared cache / fabric)
               = T_dev     if they cross a device (P2P / NVLink / xGMI)
T_push(edge) = T_sync(edge) + T_queue(N_contenders)

```

The coefficients are not guessed; phase-0 microbenchmarks fill them. Public numbers serve as seed values (section 8.5), and the first measured values from 2026-09-22 are in section 8.6 and Appendix C, including the first end-to-end split-K run, where about 1 us of queue cost per 2 us task made static 3.5-4x faster than dynamic at $n=1024$, exactly the regime the C_{queue} term is meant to capture. In the code, `MachineModel.t_sync_us(scope)` maps DOMAIN to `t_local_ns`, DEVICE to `t_cross_ns` and SYSTEM to `t_dev_ns`; on a machine without a domain level DEVICE also maps to `t_local_ns`; an uncalibrated value falls back to 200 / 800 / 10,000 ns and a note is appended to the plan log.

4.5 Honest expectations

Dense models of 30B parameters and above at batch 16 and above: 10% to 30% over vLLM / SGLang, with the ceiling set by the baseline's MBU. MoE at small batch and tensor parallelism: baseline MBU is 10% to 25%, so 1.5x to 3x has physical room, but only if levers 1 to 3 are obtained together; putting the operators into one kernel does not by itself get there. Evaluations always report MBU and a per-phase trace, never a bare multiplier.

5. System Architecture Overview

5.1 Five layers

Layer	Content	Product passed downward	Acceptance criterion
L1 Frontend adapters	Triton, TileLang, CuTe DSL, CK, hand-written HIP / CUDA; each answers only three questions: coordinate space, dependency map, resource need	TileOp + EdgeMap (hardware-independent pure data) + tile-body handle	Switching frontends changes no line below L1
L2 Event Tensor IR	Task grids, ETensor(shape, wait_count, scope, init), symbolic dimensions, data-dependent update / trigger, verification passes	Tile dependency graph with ETensors, verified acyclic and count-conserving	All verification checks green; symbolic dimensions instantiable
L3 Machine model	Exec-domain tree, visibility and atomic capabilities, capacities (LDS, registers, CUs), cost table (calibrated), lowering table; arch/gfx942.yaml and others	Read-only tables: L4 reads costs and capacities, L5 reads capabilities and lowering	New hardware is a new YAML file only
L4 Placement and scheduling passes	Tile size / resource class, affinity partitioning, event elimination by recomputation, schedule-mode selection, queue hierarchy, memory / event allocation, cross-barrier prefetch	Partition + per-subgraph mode with reason + queue layout + descriptor table	Every decision is explainable (“why static”)
L5 Backend codegen + minimal runtime	Persistent-kernel skeleton, primitive lowering (atomics / fences / sleep), linking of DSL tile bodies, kernel family (multi-device / multi-resource-class)	Kernel binary + launch parameters (event buffer, queues, descriptors, shape scalars, domain map)	Element-wise match with the reference; no deadlock

L3 is only read, never modified, by L4 and L5. At run time, per step, the host writes shape scalars and pointers, launches or posts to the resident kernel; the event tensor and the queues are the entire state; a watchdog and an event dump exist for diagnosis.

5.2 Data flow of one compile

```
Graph (symbolic) --verify--> Instance (one step's tasks and events)
  --P1--> worker counts  --P2--> domains + scopes  --P3--> smaller graph (re-P2)
  --P4--> modes  --P5--> queues  --P6--> memory  --P7--> prefetch  --verify_plan--> Plan
Plan --codegen--> megakernel_d<i>.hip + etx_lowering.h + etx_tiles.h + plan.json
Plan --sim--> deadlock check, makespan, budget
```

The kernel is per (graph, machine). plan.json is per step: shape scalars, descriptors, static queues, event counts, push lists. Shape dynamism means regenerating the plan, never the kernel.

6. Layer 1: Tile Contract and Frontends

6.1 Three questions

A megakernel compiler does three things: cut tiles, connect dependencies, order a schedule. Each needs one input, so any frontend that can answer three questions can be connected; one that cannot should not be forced in, and the operator should run as a separate kernel.

Question	Form of the answer	Notes
What is the tile coordinate space?	<code>Grid = (expr_0, expr_1, ...)</code>	Symbolic expressions allowed, e.g. <code>(B*H, ceil(S/128))</code>
What does each tile read and write?	Explicit: <code>coord -> {(tensor_slice, producer_coord)}</code> . Declarative: slice expressions for reads / writes, from which ETX infers the map	Hand-written IR gives the explicit form; Triton-like frontends give the declarative form. If inference fails (a slice that is not affine) the compiler reports an error; it never guesses
What resources does a tile need?	<code>Resource(threads, lds_bytes, vgpr, agpr, tensor_core, cluster?)</code>	Abstract need; mapping it onto a 64 KB or 160 KB LDS is L4's job

The answers form a `TileOp` containing no hardware detail. In code (`etx/ir/types.py`) a `TaskGrid` carries the grid, a `TileBody(kind, symbol, source)`, the resource, args / reads / writes, the edge maps, and optimisation annotations (`pure`, `out_to_in`, `weight_args`, `bytes_per_tile`, `duration_us`, `duration_cv`, `device`); `Resource.prefetch_bytes` is reserved for Pass 7.

6.2 Two integration modes

“DSL-agnostic” cannot remain an IR-level declaration. Tile bodies must end up in the same kernel as the persistent loop, and different DSLs produce different artefacts, so the backend offers two routes.

Link mode: the tile body is a linkable device function. The frontend compiles each tile body to a device function with a fixed ABI (`-fgpu-rdc / -rdc=true`); ETX generates the persistent loop and dispatch switch in HIP / CUDA C++ and the two are merged at link time. CUTLASS, CK, CuTe DSL (exporting device functions) and hand-written kernels take this route. Advantage: the tile body need not know ETX exists, and vendor-library-grade tiles can be inlined. Cost: limited cross-translation-unit inlining, and register allocation is the union over the whole kernel.

Host-DSL mode: the persistent loop is generated in the tile’s own DSL. Triton does not export linkable device functions, but `@triton.jit` functions can call one another, so ETX generates the loop, pop, wait and dispatch in Triton, with runtime primitives from `tl.atomic_*`, extern library functions or `inline_asm` (Triton-Distributed has shown the route is viable). Advantage: one compilation unit, optimised as a whole. Cost: one skeleton template per DSL, and the DSL’s expressiveness limits the skeleton (Triton has no warp specialisation).

6.3 Tile ABI (link mode)

```
// Fixed signature of every tile body; the frontend exports to it
struct etx_ctx {
    int32_t      coord[4];    // task coordinate (symbolic shape already instantiated)
    const int32_t* shape;     // this step's symbolic scalars, in graph.symbols order
    void* const*  args;       // tensor pointer table, indexed by plan.json "args"
    etx_event*    events;     // event buffer base (int32 counters)
    const int32_t* ev_offset; // per-event-tensor offset into events
    const int32_t* ev_shape;  // per-event-tensor shape, 4 ints each
    uint32_t      domain;     // exec domain and worker id (discovered at launch by L5)
    uint32_t      worker;     //
    void*         lds;        // this worker's LDS base, sized by resource class
};
extern "C" __device__ void etx_tile_<name>(const etx_ctx*);

// Runtime primitives a tile body may call (defaults provided; a frontend may override)
__device__ void    etx_wait_<SCOPE> (etx_event*);        // SCOPE in DOMAIN | DEVICE |
SYSTEM
__device__ int32_t etx_arrive_<SCOPE>(etx_event*);        // returns remaining count
__device__ void    etx_push (const etx_queue*, int32_t task);
__device__ int32_t etx_try_pop(const etx_queue*, int32_t* ticket);
ETX_BACKOFF();                                           // s_sleep / nanosleep
ETX_DOMAIN_ID();                                         // HW_ID.XCC_ID / %smid / 0
```

The ABI deliberately exposes six things: coordinate, scalars, pointer table, events, position, LDS. A tile body that needs more is attempting to schedule and belongs in L4.

6.4 Frontend matrix

Frontend	Mode	Dependency form	Notes
Triton (NVIDIA / AMD backends)	host-DSL	declarative (slice inference)	No warp specialisation, no TMA multicast control; suited to GEMV, norm, gather / scatter tiles
TileLang / TVM TensorIR	either	declarative	The paper's implementation path; symbolic shapes already supported
CuTe DSL / CUTLASS	link	explicit	Main GEMM and attention tiles; Hopper / Blackwell features fully available
Composable Kernel (AMD)	link	explicit	Main AMD GEMM / attention tiles; MFMA and LDS layout handled inside CK
Hand-written HIP / CUDA	link	explicit	fleet-mi300x's GEMV and MLA tiles are of this kind

In the repository, `etx/frontends/tileop.py` defines the contract, `linkmode.py` emits the link-mode prototypes header, and `triton_host.py` emits a Triton skeleton (not yet runnable). `TileBody.kind` is one of `hip_link`, `cuda_link`, `triton`, `builtin`.

7. Layer 2: Event Tensor IR

7.1 Core types

```
Tensor    = (shape[sym|const], dtype, layout_hint)
ETensor   = (shape[sym|const], wait_count[sym|const|runtime], scope, domain_id, init_kind)
TaskGrid  = (grid[sym], tile_ref, resource_class, in_edges{ETensor: map}, out_edges{ETensor:
map},
            pure: bool, out_to_in: map)      # pure + output-to-input map: precondition for
Pass 3 inlining
Step      = (graph_fn, shape_scalars, arg_table)  # one call = one iteration
```

- `wait_count`: number of notifies before the event reaches zero. A constant, a symbol (`WORLD_SIZE`), or a runtime value (tokens routed to expert *i*); a runtime value must be written by an upstream tile.
- `scope`: visibility range: `workgroup < cluster < domain < device < system`. `domain` is the level ETX adds relative to the paper; it corresponds to an XCD, GCD or die. The paper's events have one scope (`device`), which is fine on a homogeneous L2 and an order of magnitude of cost on a chiplet part.
- `domain_id`: the exec domain whose tasks this event mainly serves. Filled by the L4 partitioning pass; empty in L2.
- `init_kind`: `static` (constant), `per_step` (host writes the counts each step, e.g. batch-dependent counts), `runtime` (a tile writes the counts, e.g. MoE routing counts). In code: `InitKind.STATIC / PER_STEP / RUNTIME`, with `runtime_init_by` naming the writing grid.
- `epoch`: optional flag for resident kernels (section 11).

Graphs keep grids in program order, which is required to be a topological order.

7.2 Four edge-map forms

Form	Example	Use	What the compiler can analyse
Affine (einsum subset)	"ij->i"	Reductions, splits, static dependencies	Exact fan-in / fan-out; can be statically queued
Index arithmetic	"ij->(i/2)j"	Mismatched tile sizes (an RS tile is twice an MM tile)	Same
Runtime range	"i->range(indptr[i], indptr[i+1])"	One producer triggers a variable number of consumers (MoE expert to GroupGEMM tiles)	Only the upper bound; static scheduling can queue by the bound and rely on <code>wait_count</code>
Runtime gather	"i->topk[i,:]"	Which events a producer notifies is data-dependent	Only the fan-out <i>k</i> ; counts are initialised by the <code>init_kind=runtime</code> tile

The constraints are the precondition for analysis: maps must be affine, or `range(affine, affine)`, or `gather(tensor)`; arbitrary control flow is forbidden in edge maps; runtime-value tensors must come from an upstream tile's writes with release semantics. Pass 3 additionally uses the special map `"*"` (all coordinates) when a consumer inherits a removed producer's in-edges.

7.3 Semantics (part of the IR, not a backend detail)

```
arrive(ev) == release-fence; atomic_dec(ev.count)    -> ev fires when the result is 0
wait(ev)    == spin_until(ev.count == 0); acquire-fence
trigger(ev) == when arrive returns 0, push ev's consumer task set to the scheduler
(dynamic / hybrid only)
```

The producer's writes must be visible to consumers before `arrive` (release); the consumer's reads must follow the return of `wait` (acquire); and both hold at the event's scope. If the backend finds the target cannot provide these semantics at that scope (the atomic pitfalls in section 8.4), it must fail at compile time, never downgrade silently.

7.4 Six verification checks

All six must be green before a graph enters L4. Checks 1-4 concern dependency semantics and run in `etx/ir/verify.py`; checks 5-6 need placement results and run in `etx/passes/plan.py::verify_plan`, which also checks static-queue order.

Check	What it checks	Consequence of omitting it
Acyclic	The task-level dependency graph has no cycle for any value of the symbolic dimensions (in code: every in-edge event is produced by an earlier grid in program order, and no grid reaches itself through events)	Deadlock that is very hard to locate on the device
Count conservation	<code>wait_count</code> equals the actual fan-in (runtime ranges use the upper bound, and the initialising tile must be in the graph and precede every consumer)	Early trigger (wrong result) or never trigger (deadlock)
Runtime values first	The tile that writes <code>indptr</code> / <code>topk</code> -class tensors is strictly earlier on the dependency graph than every <code>arrive</code> / <code>trigger</code> that reads them, and an event path orders them	Stale routing is read; MoE results are wrong but look plausible
Unique tile coverage	Every output element is written by exactly one tile (in code: exactly one writer grid per tensor; per-element coverage is a frontend obligation)	Silent data race
Scope monotonicity	An edge crossing domains uses an event scope of at least <code>domain</code> ; crossing devices at least <code>system</code> ; any event of a grid scheduled through the device-global queue is at least <code>device</code> (section 9.4)	Visible locally, invisible remotely
Co-residency satisfiable	The persistent kernel's worker count is at most the target's co-residency limit for the resource class	Workers not co-resident wait for a producer that is never scheduled

8. Layer 3: Machine Model

8.1 Exec-domain tree

The paper's IR is hardware-independent but its implementation and evaluation ran on one machine. Extending to other hardware is not a matter of adding backend branches; the shape of a machine is written as a tree and a few tables, and L4 / L5 only read them.

Machine	Tree	Notes
MI300X (gfx942)	device (one HIP device) -> 8 XCDs	38 CUs per XCD, private 4 MB L2 per XCD; cross-XCD only through the 256 MB Infinity Cache; LDS 64 KB, wave64, 5.3 TB/s. domain = XCD, 8 of them
MI250X (gfx90a)	device 0 -> GCD0 (110 CUs); device 1 -> GCD1 (110 CUs)	The two GCDs are two HIP devices; 8 MB L2 per GCD; no Infinity Cache; floating-point atomics do not cross Infinity Fabric. One card = two kernel instances
B200 (sm_100)	device (2 dies, one to software) - > 148 SMs, optional clusters, unified L2	DSMEM, TMA multicast, multimem reduction; 228 KB smem per SM; wave32; latency difference between dies but coherent. domain = whole card (optional die affinity)
MI355X (gfx950)	device -> 8 XCDs	32 CUs per XCD, private 4 MB L2; LDS 160 KB; 8 TB/s HBM3E; same tree shape as MI300X with different capacities: the same YAML with different numbers

ETX recognises only this tree. Within one domain an event goes through the local L2 at cost `T_local`; across domains through the shared cache / interconnect at `T_cross`; across devices through P2P / NVLink at `T_dev`. New hardware is a new tree, three cost coefficients and one capability table; RDNA (WGPs, wave32, no shared-cache partitioning) is just another tree shape, and a chiplet is one more tree level, not a special case. Because MI250X's two GCDs are two devices, "one card" is two kernel instances sharing system-scope events, on the same code path as multi-GPU tensor parallelism.

In code (`etx/machine/model.py`), `effective_scope()` collapses scopes a machine does not distinguish: CLUSTER becomes DOMAIN without `cluster_launch`; DOMAIN becomes DEVICE on a machine with a single domain; WORKGROUP becomes DOMAIN or DEVICE.

8.2 Capability table versus cost table

The capability and visibility blocks decide correctness; the cost block decides performance. The two are never mixed. The `gfx942` file (Appendix A) has three visibility rows: domain (`release s_waitcnt vmcnt(0)`, since L1 is write-through and within one L2 only the consumer's L1 needs invalidating; `acquire buffer_inv sc0`; any memory), device (`release buffer_wbl2 sc1; s_waitcnt vmcnt(0)` to write back the producer XCD's dirty L2 lines; `acquire buffer_inv sc1`; any memory) and system (`release buffer_wbl2 sc0 sc1`; `acquire buffer_inv sc0 sc1`; fine-grained memory, since coarse-grained silently downgrades to device scope). Its capability rows record, among others, no cluster launch or DSMEM (`gfx1250` introduces cluster scope; LLVM degrades unsupported cluster scope to agent scope), no `multimem` and no switch (eight GPUs are a point-to-point xGMI mesh), dword-only direct-to-LDS loads (`gfx950` widens to 128 bit per lane), FP atomics over Infinity Fabric (false on MI200), `wg_to_domain_map: discover`, cooperative launch that returns `hipErrorCooperativeLaunchTooLarge` rather than deadlocking, and `s_sleep` backoff (64 x N cycles, up to about 8000; `s_setprio` 0-3; `s_memrealtime` at 100 MHz).

The critical row is `poll`. A plain load hits the non-coherent per-CU L1 (wave scope is "Hit LRU") and never sees another CU's write; event polling must use a load with the `sc1` bit, an atomic load, or an L1 invalidate followed by a load. Kog wrote inline assembly for this because `__hip_atomic_load` cannot do a 3-dword read. Facts of this kind can only exist as data in a table, not as something each tile author must remember.

8.3 Portability matrix

Field	MI250X (gfx90a)	MI300X (gfx942)	MI355X (gfx950)	RDNA4 (gfx12)	H100 (sm_90)	B200 (sm_100)
Exec domain	2 GCD x 110 CU, each a HIP device	8 XCD x 38 CU = 304 (40 per XCD, 2 disabled)	8 XCD x 32 CU = 256 (36 per XCD, 4 disabled)	WGP array, single domain	132 SMs, single domain (L2 in two partitions, far hit 414 vs near 258 cycles)	148 SMs (74 per die), single domain
L2 ownership	8 MB per GCD	4 MB per XCD; cross-XCD via wbl2 / inv plus MTYPE probing	4 MB per XCD, 16 channels	Unified	50 MB, two partitions	126 MB, die crossing about 30 cycles
Shared cache	None (3-level latency)	256 MB Infinity Cache, memory-side, about 218 ns	256 MB, 2 IODs	Infinity Cache (consumer)	-	-
LDS / smem per CU	64 KB	64 KB	160 KB, double read bandwidth	64 KB per CU, 128 KB shared in WGP mode, 64 KB per workgroup	228 KB	228 KB
Wave	64	64	64	32 (WMMA wave32 only)	32	32
Cluster / DSMEM	None	None	None	cluster scope from gfx1250	Yes; portable limit 8, max 16; DSMEM 181-213 cycles vs global about 1110	Yes
Async copy to LDS	None	32 bit per lane only	128 bit per lane, with transposing reads	gfx11 none; gfx1250 TDM	TMA / cp.async	TMA + multicast
Multicast reduction write	None	None	None	None	multimem (NVLink SHARP)	multimem
FP atomics over the fabric	No (NOP risk)	Yes (fp32 add, packed fp16 / bf16, fp64 add / min / max)	Yes	-	Yes	Yes
Inter-card link	4 IF links between GCDs, 200+200 GB/s; kernel-initiated P2P 43-172 GB/s, latency 8.7-18.2 us	7 xGMI links, 64 GB/s per direction per link, about 48 measured; point-to-point mesh, no switch	7 IF links, aggregate over 1 TB/s	PCIe	NVLink 900 GB/s + NVSwitch	NVLink 5 + NVSwitch

Field	MI250X (gfx90a)	MI300X (gfx942)	MI355X (gfx950)	RDNA4 (gfx12)	H100 (sm_90)	B200 (sm_100)
Backoff / timer	s_sleep	s_sleep (64 x N cycles), s_setprio, s_memrealtime	same	same	nanosleep (up to 2t, cap 1 ms), griddepcontrol	same
Bandwidth	3.2 TB/s	5.3 TB/s	8 TB/s, 288 GB HBM3E	about 0.6-0.9 TB/s	3.35 TB/s	8 TB/s

Note the FP-atomics row: on MI200 a floating-point atomic cannot cross Infinity Fabric, and if the target is host memory and PCIe does not support the atomic the hardware degrades to load-op-store or, in the extreme, a NOP. This is not a performance problem; it is a silent error, so “does this atomic’s semantics hold on the target memory” must be a compile-time-queryable capability bit.

8.4 Three silent-error pitfalls (AMD atomics)

1. A system-scope atomic on coarse-grained memory is downgraded to device scope. Event tensors used for cross-device synchronisation must be allocated fine-grained.
2. When PCIe does not support an atomic, the GPU side degrades to load-op-store; all waves committing to that address stall together and the CPU does not see an atomic sequence.
3. Unsupported floating-point atomics on uncached memory may be NOPs on MI200: wrong results, no error.

Conclusion: the event-tensor allocator is part of L4, not a runtime detail; it chooses the memory type by scope and asserts legal combinations at compile time. This is the direct reason scope is in the IR. In code, Pass 6 sets `ep.memory = machine.memory_for(scope)`, which raises if the lowering table lacks the scope, and logs when a system-scope event lands on a machine without `fp_atomic_over_fabric` (integer counters only).

8.5 Cost table seed values (public measurements)

Measurement	MI300X	H100 / B200	Design implication
Atomic round trip between two workgroups, same vs cross XCD	116 ns vs 202 ns (Chips and Cheese)	B200 about 512 atomics per cycle over the whole card	A cross-domain edge is at least 1.7x more expensive and occupies Infinity Fabric
Agent-scope release / acquire fence	115 / 137 ns	-	Every arrive pays at least one fence; last-arriver merges the write-back
Cross-XCD one-way flag latency	about 703 ns	DSMEM 181-213 cycles vs global about 1110 cycles	A cross-XCD event on MI300X is of order 1 us; ten per layer is 1% of token time
Persistent-kernel phase switch: counter + fence vs sentinel sc1 poll	7.6-7.9 us vs 0.80-0.93 us (Kog, 256 CUs)	-	Whole-card counter zeroing is expensive; a sentinel is a candidate lowering with the event-tensor abstraction unchanged
XCD-aware program-id remap (AMD)	L2 misses about 5M -> 3.1M, +67 TFLOPS	-	Placement alone is worth about 10%
L2 / Infinity Cache / HBM latency	81-108 / 218-258 / 342 ns	H100 L2 near 258, far 414 cycles	The cache level holding the counter sets the minimum wait period

fleet-mi300x is consistent with this: reducing global events from 10 to 3 per MoE layer was worth about 1 ms per token over 27 layers, and a cross-XCD event cost more than an order of magnitude above an intra-XCD one.

8.6 Measured calibration values (2026-09-22, MI300X, ROCm 7.2.4)

The following were measured on a rented Hot Aisle 1x MI300X VM with the repository's `bench/calib` programs; the full table is in Appendix C. They are the first real entries for the `costs: block` of `gfx942.yaml`, and each corrects or confirms a seed assumption.

- Workgroup-to-XCD mapping: workgroup k landed on XCD $(k+6) \bmod 8$ (first sixteen: 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5), neither the documented round-robin from XCD 0 nor the $(k+4) \bmod 8$ reported publicly. This confirms `wg_to_domain_map: discover: read HW_ID.XCC_ID` at kernel start; never assume the formula.
- One-way arrive latency (arrive is an agent-scope atomic RMW). DEVICE protocol (producer `buffer_wbl2_sc1`, consumer agent-scope atomic-load poll): 873 ns same XCD, 892 ns cross XCD. INVL1 protocol (no write-back; consumer `buffer_inv_sc0` then a plain load): 642 ns same XCD, 742 ns cross XCD, observing the arrive correctly in both cases. A plain or `sc0` load without the invalidate never observes the arrive; it spins forever on the stale per-CU L1. Implications: (a) polling counters through L2 after an L1 invalidate is correct and about 25% cheaper than agent-scope atomic loads; (b) the same-versus-cross-XCD difference for counters is about 100 ns, far smaller than the public pure-atomic figures (116 vs 202 ns) suggested, because every RMW executes at the fabric, so the domain-scope advantage lies mostly in skipping the producer's L2 write-back of payload data, not in the counter; (c) plain loads are never acceptable for flags. The helper `etx_amdgcn_poll_l2()` implements the INVL1 poll; the v0.3 visibility table still lists the atomic load as `poll`, and switching the domain and device rows is a one-row table change scheduled with the phase-1 bring-up.
- One-way flag latency with payload, timed with the global 100 MHz `s_memrealtime` counter (producer stamps payload, `buffer_wbl2_sc1`, agent-scope store; consumer agent-scope poll plus `buffer_inv_sc1`): 951 ns same XCD, 916 ns cross XCD, 0 stale payloads in 10,000 iterations each. The lowering table's release / acquire sequences are correct; the 703 ns seed for `t_flag_cross_ns` becomes about 0.92 μ s.
- Fence costs: DEVICE release (`buffer_wbl2_sc1 + s_waitcnt`) 181 ns; DEVICE acquire (`buffer_inv_sc1`) 120 ns; DOMAIN release plus acquire (`s_waitcnt + buffer_inv_sc0`) 44 ns together. A domain-scope edge saves about 257 ns of fences per arrive / wait pair, in addition to the payload write-back.
- Queue contention, 65,536 pops of one ring by N workgroups, aggregate million pops per second. CAS-based pop (v0.1 design): $N=1$ 0.89, $N=8$ 1.85, $N=64$ 1.10, $N=512$ 0.41; it collapses past 8 poppers. Ticket-based pop (v0.2 design: one `atomicAdd` on head per pop, then the popper polls its own slot; one `atomicAdd` on tail plus one `atomicExch` per push): $N=1$ 1.15, $N=8$ 8.48, $N=64$ 47.6, $N=128$ 65.0, $N=256$ 55.2, $N=512$ 76.6. The uncontended pop costs about 870 ns. The v0.2 runtime uses the ticket ring, and the `C_queue` term of the schedule-mode formula is fed by this curve: per-popper latency grows roughly as N divided by aggregate throughput, about 2 μ s at 128 poppers and 4.6 μ s at 256.
- Toolchain: the generated split-K and MoE persistent kernels pass `hipcc -fgpu-rdc -fsyntax-only` for `gfx942` and the 46 tests pass on the VM.
- First end-to-end execution. After the calibration runs, the split-K row-sum example (`examples/splitk_sum.py`, tiles `examples/tiles/splitk.hip`, host `examples/hosts/splitk.hip`) was compiled with `hipcc -fgpu-rdc`, launched cooperatively with 608 workers (8 XCDs x 76 workgroups, 2 per CU, 256 threads) and matched the CPU fp32 reference with max abs error $4.8e-7$ under all three scheduling modes. Per-step time, best of 30-50 repetitions (the first launch, about 21 ms, includes code load): $n=64$ (320 tasks, 64 events) static 0.101 ms, hybrid 0.115 ms, dynamic 0.118 ms; $n=1024$ (5120

tasks, 1024 events) static 0.128 ms, hybrid 0.441 ms, dynamic 0.512 ms. Tasks are about 2 us and the queue path costs about 1 us per pop plus a push, so on tiny tasks static wins by 3.5-4x, consistent with the `C_queue` term of the schedule-mode formula and with the paper's static-versus-dynamic results on regular workloads. Two defects were found only on hardware and are described in sections 9.4 and 10.3.

9. Layer 4: Placement and Scheduling Passes

Seven passes run in order. Each reads only the L3 tables, and each writes data into the plan and a line into the plan log; none changes IR semantics.

9.1 Pass 1: tile size, resource classes, worker count

Input: the abstract resource need of every `TileOp` and the target capacities. Output: workgroups per CU, workers per domain, kernel instances. Tile shapes are the frontend's business; the pass checks that they fit (LDS need including `prefetch_bytes` within `lds_kb`, `vgpr` + `agpr` within `regs_per_lane`, raising otherwise) and derives the co-resident workgroups per CU from registers, LDS and `max_wg_per_cu`, which fixes the persistent grid: `workers_per_domain = cus_per_domain x wg_per_cu`. A 64 KB versus 160 KB LDS changes the feasible tile size, so the same operator has different tile counts on `gfx942` and `gfx950` and the event tensor's shape changes with it; this is why shapes stay symbolic.

The pass also addresses the register-union problem the paper does not mention: a kernel's VGPR / LDS footprint is the maximum over all tile types, so the heaviest tile sets the occupancy of the whole kernel. ETX groups tiles into resource classes (`rc<N>`, `N` being the workgroups per CU the tile permits); tiles of one class go into one persistent-kernel instance, several instances are co-resident on different streams with a share of the CUs each, and all share one event tensor. This kernel-family mechanism is the same code as multi-device instances (section 10.4). Splitting is optional and off by default: on fleet-mi300x a 343-register union caused no loss for the bandwidth-bound GEMV, so the cost model decides.

LDS is likewise not allocated worst-case but cut into fixed-size pages (Hazy on H100: 13 pages x 16 KB). Tile bodies request and release pages from a page allocator so the next tile's weight load can start while the current tile is still writing back. The page count follows from the resource class and `lds_kb`: `gfx942` 4 pages, `gfx950` 10, `sm_90` 13.

9.2 Pass 2: event-affinity partitioning (the chiplet-aware core)

Problem: partition the task / event graph into `k` blocks (`k` = number of domains) minimising the total weight of crossing edges, where edge weight = fan-in / fan-out count x `T_sync` of the edge. The first version is greedy with local improvement:

1. Seed with the operator of largest fan-in / fan-out and put it and its events in one domain (MoE's GroupGEMM with its expert events; attention's heads with their merge).
2. Consecutive tile coordinates land in the same domain, with a `GROUP_SIZE_M`-style swizzle on top to preserve L2 locality.
3. When cross-domain edges exceed budget, promote the event to the shared cache as the meeting point, or convert the edge to a statically ordered long queue (no push).

Output: `domain_id` per task, and scope plus storage type per event (plain VRAM / shared-cache resident / cross-device fine-grained). In fleet-mi300x's MoE layer this step together with Pass 3 reduced global events from 10 to 3; the remaining 24 are XCD-local.

In code (p2_affinity.py): grids without dependencies are chunked contiguously across domains; dependent tasks follow the majority domain of their producers unless that domain is loaded beyond options.affinity_imbalance times the mean; every event then receives the lowest scope that is still correct (DOMAIN if all parties share a domain, DEVICE if they share a device, SYSTEM otherwise), passed through effective_scope(); cross-domain and cross-device edge counts are recorded per event.

9.3 Pass 3: event elimination by recomputation (lever 1)

An event exists because a producer's output is consumed by many tasks. If the producer is a pure function, its inputs have already been synchronised by an earlier event, and it is cheap, then every consumer recomputes it in its own prologue and the event and the producer task are deleted. The price is redundant parallel microseconds; the return is serial synchronisation microseconds.

Example (fleet-mi300x): before elimination, x (4 KB) goes through one RMSNorm task and one global event E1 to 296 GEMV workers; phase A has 1 worker busy and 295 idle, followed by about 1 us of cross-XCD round trip, drain and ramp. After elimination, each of the 296 GEMV workers computes RMSNorm(x) in its prologue (4 KB read from L2); each worker takes about 1 us longer, in parallel, so the phase grows by 1 us while the event, the task and the drain disappear. Redundant bytes: $296 \times 4 \text{ KB} = 1.2 \text{ MB}$, all L2 hits. Compilers call this rematerialisation; HPC calls it communication-avoiding; here the saving is synchronisation. At batch 1 every phase is already hundreds of workers doing the same small thing; redundancy is parallel and the event is serial, so redundancy almost always wins.

Three variants:

Variant	Method	Removes	Redundancy cost
Prologue recomputation	Consumers recompute a small producer (norm, RoPE, KV post-processing, q-absorb, router top-k)	Producer task + event	consumers x producer cost, in parallel
Consumer-side folding	No reduce task; each consumer sums the k partials in its prologue	Reduce task + event	consumers x k partial reads (fleet: $296 \times 8 \times 4 \text{ KB}$ through L2 / Infinity Cache)
Last-arriver compute	The producer whose arrive reaches zero performs the reduction	Reduce task; the event stays	Reduction becomes serial; fleet measured about 4 us per fold point, slower; off by default

Design inequality:

```

gain = T_sync(E) + T_drain(producer phase) # sync round trip + the stretch
with few workers active
cost = max over consumers ( bytes_P / BW_cache + compute_P ) # parallel: maximum, not sum
      + [P's inputs not in L2] x consumers x bytes_P / BW_HBM # then a sum, usually a veto
rewrite if P.pure and P.in_events subset of events C has already waited on
      and P.out_to_in maps under C's coordinates
      and gain > cost
      and (bit-exact mode) P's floating-point summation order matches the reference

```

Three cases in which it must not fire are logged: the producer would re-read weights from HBM (q-absorb is the boundary case: 2.1 MB more per layer, worthwhile only because chunks of one head sit on one XCD and hit L2); reductions with more than about 8 partials; and too many consumers with inputs not in cache. The log prints the removed events and the redundant bytes bought, so a regression can be traced to the recomputation that blew the cache.

The implemented rule (p3_event_elim.py) is a conservative form of the above. For each event with exactly one producer P and at least one consumer:

- P must be marked pure and carry out_{to_in}; no consumer may be on a different device from P; P's outputs may be used only by those consumers and must not appear in any runtime edge map (runtime tensors must be produced exactly once).
- $\text{gain} = t_{\text{sync_us}}(E.\text{scope}) + P.\text{duration_us} \times (1 - \min(1, n_P / \text{total_workers}))$, the second term being the drain of a phase in which only n_P of the workers are busy.
- $\text{cost} = P.\text{duration_us}$; if P reads weight tensors and $\text{bytes_per_tile} \times \text{consumers_per_domain}$ exceeds the leaf domain's L2, $\text{bytes_per_tile} \times n_{\text{consumers}} / \text{BW_HBM} / \text{total_workers}$ is added and the log notes how many MB would be re-read from HBM.
- The rewrite fires only if $\text{gain} > \text{cost}$. Each consumer's prologue receives P's prologue chain followed by P; the consumer drops the eliminated in-edge and inherits P's in-edges with the "*" map (waiting for everything P would have waited for, correct for any coordinate mapping); P's reads and args are merged in; the consumer's duration grows by cost . The producer grid and the event are removed, P's outputs are re-labelled recomputed, and the plan records the eliminated event. The pipeline then re-instantiates and re-runs Pass 2.

9.4 Pass 4: schedule-mode selection

```

S_balance = straggler time dynamic scheduling is expected to recover (from load variance or
a profile)
C_cross   = cross-domain pushes x T_cross (x T_dev across devices)
C_queue   = queue-contention cost (calibrated curve, rising with the number of contenders)

dynamic   if S_balance > C_cross + C_queue and there are no cross-device edges
hybrid    if the domain has irregular load and there are many cross-domain edges
static    otherwise

```

The empirical thresholds start from the paper's two data points: MoE (same domain, irregular routing, 1024 tokens) dynamic 1.08 vs static 1.04, choose dynamic; dense TP=4 (cross-device) dynamic 0.83 vs static 1.09, choose static. As cross-domain edges multiply, the gain of dynamic scheduling is consumed by push cost.

The implemented rule (p4_schedule_mode.py), per task grid g with n task instances:

- $\text{waves} = \text{ceil}(n / \text{total_workers})$; cross_dom and cross_dev count, over all of g's tasks and in-edge producers, the producer-consumer pairs in different domains or different devices.
- $S_balance = \text{duration_cv} \times \text{duration_us} \times \text{waves} \times 2.0$.
- $C_cross = \text{cross_dom} \times t_{\text{push_us}}(\text{cross_domain}=\text{True}) / \text{total_workers} + (\text{waves} \times t_{\text{sync_us}}(\text{DEVICE}) \text{ if } \text{cross_dom} > 0 \text{ else } 0)$, where $t_{\text{push_us}}(\text{cross})$ is $t_{\text{push_ns}}$ plus the device-scope sync cost.
- $\text{sharing} = \text{total_workers}$ if no cross-domain edges (one device-global queue) else $\text{workers_per_domain}$ (domain-local queues); $C_queue = \text{waves} \times t_{\text{pop_us}} \times (1 + 0.01 \times \text{sharing})$.
- Decision order: a forced mode from options; else $\text{cross_dev} > 0$ gives static ("pushes over P2P are prohibitive, ETC TP=4 dynamic 0.83x"); else a grid with runtime edges gives hybrid if $\text{cross_dom} > 0$ else dynamic (data-dependent grids never go static, since static would degrade to an E[0] barrier); else if $S_balance > \text{MARGIN} \times (C_cross + C_queue)$ with $\text{MARGIN} = 1.5$, dynamic if $\text{cross_dom} == 0$ else hybrid; else static.

The 1.5x margin biases the decision toward static because the paper's regular workloads lose 6% to 17% under dynamic scheduling; dynamic must be predicted to win clearly before it is chosen. The first hardware run confirmed the bias in the other direction as well: on the split-K example with about 2 us tasks, static ran the n=1024 step in 0.128 ms against 0.441 ms hybrid and 0.512 ms dynamic (section 8.6).

A scope rule found on hardware is attached to this pass. A grid scheduled through the device-global queue runs its tasks on any XCD, so the DOMAIN scope that Pass 2 assigned to its events from placement is wrong: on the first dynamic run consumers read stale L2 and produced zeros while the kernel “completed”. Pass 4 therefore raises every event produced or consumed by a dynamically scheduled grid to DEVICE scope, and `verify_plan` rejects a plan that violates this. Hybrid mode does not need the rule because its pushes and pops stay inside the domain.

The reason string with the computed terms is stored in `plan.reasons[grid]` and printed, for example:

```
subgraph moe.group_gemm : hybrid    (S_balance=+7.6%, intra-domain dynamic, 3 cross-domain
edges -> static)
subgraph attn.qkv_rope  : static    (S_balance~0, 0 cross-domain edges)
subgraph tp.gemm_rs      : static    (cross-device edges=18, C_cross=2.1ms > S_balance=0.4ms)
```

With `t_pop_ns` set to the measured uncontended 870 ns, the linear contention factor reproduces the measured per-popper latency of the ticket ring at 128 poppers (about 2.0 us both ways) and underestimates it by about 30% at 256 poppers (3.1 us predicted, 4.6 us measured); replacing the linear factor by the measured curve is a cost-table change, not a pass change.

9.5 Pass 5: queue hierarchy (the mechanism of hybrid scheduling)

Hybrid scheduling is not a new mechanism but a different assembly of the same primitives: push / pop inside a domain, arrive / wait plus pre-ordered queues between domains.

- Each domain has a local ready queue and its workers; pushes inside the domain cost `T_local`. Local events (`scope = domain`) live in that XCD’s L2.
- Each worker also has a statically pre-ordered long queue holding the static segment; cross-domain consumer tasks live there and rely on `wait`, not `push`.
- Global events (`scope = device`) live in the shared cache; each cross-domain edge pays one `arrive` at `T_cross`.
- There is no cross-domain push. Intra-domain irregularity (MoE routing) is absorbed by the local queue; the number of cross-domain edges is minimised by Pass 2 and queued statically. The global queue holds only tasks of pure-dynamic grids whose consumers cross domains, and its entry count is bounded by the number of such tasks, so the centralised queue’s contention is removed structurally.
- Across devices the global event lives in fine-grained memory and the shape is identical: MI250X’s two GCDs and 8-card TP are the same path.

In code (`p5_queues.py`): static tasks are assigned round-robin to the workers of their domain in program order, so producers always precede consumers in every worker queue; local queue capacity per (device, domain) is the number of dynamic / hybrid tasks there plus 16; the global queue capacity per device is the number of pure-dynamic tasks plus 16; consumer lists per event coordinate are materialised for pushes (the in-kernel inverse-map alternative, MPK-style ranges, is not implemented). The design page carries an interactive discrete-event illustration of the same trade-off (variance makes static lose to dynamic; cross-domain push cost makes dynamic lose to hybrid); it is an illustration, not a prediction.

9.6 Pass 6: memory planning and event allocation

- Intermediate tensors: lifetime and placement; an intermediate consumed within one domain is placed where that domain’s L2 can hold it (fleet-mi300x keeps each layer’s 1.2 MB compressed KV cache in the XCD’s 4 MB L2 for the whole layer). In code a tensor read and written within one domain is `domain_local`, within one device `device`, otherwise `fine_grained`; weights are `weights_hbm`.

- Event tensors: memory type by scope (plain / uncached / fine-grained), asserted at compile time against L3's visibility table.
- Static path: each worker's pre-ordered queue is materialised in global memory; descriptors are fixed-length records (subgraph_id, coord, task_type) with no pointer chasing (etx_task in the ABI: type plus coord[4]).

9.7 Pass 7: cross-barrier weight prefetch (lever 3)

Barriers block activations. Weights depend on no event and are statically known, so while a worker spins on an event it can already stream its share of the next phase's weights into LDS or registers, and compute the moment the event fires. The effect is that each barrier's fixed latency is covered by the next phase's bytes.

No overlap:	[phase k compute][wait for event: idle	][phase k+1 load weights]
	[compute]	
Overlap:	[phase k compute][wait for event: load k+1 weights]	[compute]

The pass does three things:

1. The next task of a static segment is known: the worker's pre-ordered table names it, so the compiler computes its weight-slice addresses into the current task's epilogue; in dynamic segments early push also makes it known, only later.
2. Where the bytes land is a capability. gfx942 has no TMA and direct-to-LDS is 32 bit per lane, so prefetch goes to VGPRs / AGPRs or only warms L2; gfx950 has 128-bit-per-lane direct-to-LDS; Hopper / Blackwell use TMA with an mbarrier. This is the canonical "capability decides the method" case and cannot be one generic code path. In code the method is tma, lds (async_copy_to_lds == wide), l2_warm (dword_only), or the pass is skipped with a log line.
3. It is decided together with resource classes and LDS paging, because prefetch consumes registers and LDS pages and can squeeze the current tile's occupancy. fleet-mi300x once added a --prefetch-next option alone and lost 4% from register pressure and L2 pollution; Hazy reached 78% MBU because paging and prefetch were designed together. Pass 1 therefore counts the prefetch buffer as a resource need, and the cost model enables prefetch when T_fixed(barrier) - occupancy loss caused by prefetch is positive. In code, entries whose lds_bytes + prefetch_bytes exceed the LDS budget under the lds method are skipped and counted.

Pass 7 and Pass 3 are complementary: Pass 3 deletes the barriers that can be deleted; Pass 7 uses up the waiting time of those that cannot.

10. Layer 5: Lowering, Runtime and Generated Kernel

10.1 Primitive x architecture lowering table

Primitive	NVIDIA sm_90 / sm_100	CDNA3 / CDNA4	CDNA2
arrive, domain scope	(no domain level; degrades to device)	s_waitcnt vmcnt(0) (L1 is write-through) + agent-scope atomic; the counter lands in this XCD's L2; no write-back fence (Fleet and fleet-mi300x both observed 0 stale words)	Same, within the GCD

Primitive	NVIDIA sm_90 / sm_100	CDNA3 / CDNA4	CDNA2
arrive, device scope	fence.acq_rel.gpu + red.global.gpu.add	buffer_wbl2 sc1; s_waitcnt vmcnt(0) + atomic (all RMWs forward to Infinity Fabric); the producer's L2 write-back cannot be skipped, but N producers on one XCD can have the last arriver write back once	Same
arrive, system scope	red.global.sys.add	buffer_wbl2 sc0 sc1 + atomic; the event must be in fine-grained memory (hipExtMallocWithFlags) or it silently downgrades	Integer atomics only; no floating-point atomics
wait	ld.acquire.gpu spin + nanosleep backoff; no spin if a hardware tile trigger exists	load sc1 (device) or load sc0 sc1 (system) spin + s_sleep; after return buffer_inv sc0 (domain) / sc1 (device) / sc0 sc1 (system). A plain load hits stale L1: the most common silent error	Same
Low-cost intra-group sync	cluster barrier + DSMEM (181-213 cycles)	None; degrades to a domain event; gfx1250 introduces cluster scope	None
Reduction write (communication fusion)	multimem.ld_reduce	No equivalent and no switch: P2P store / atomic accumulate into the peer's fine-grained HBM (IPC-handle mapping), or owner-side L2-resident reduction (to be tested)	None (and FP atomics do not cross IF)
Co-residency guarantee	cudaLaunchCooperativeKernel	hipLaunchCooperativeKernel: CLR computes the co-residency limit from occupancy and returns hipErrorCooperativeLaunchTooLarge when exceeded (better than deadlock); ROCm's own grid.sync is a device-scope atomic plus s_sleep(1) polling	Same
Domain discovery	SM id from %smid; die affinity optional	Read HW_ID.XCC_ID at start and build the worker-to-domain table; never trust wgid % 8 (measured (k+6) mod 8 on 2026-09-22; (k+4) mod 8 reported publicly before)	Device id is the domain
LDS page allocation	213 KB smem in 13 pages x 16 KB (Hazy); pages guarded by mbarrier	64 KB in 4 pages x 16 KB; gfx950 160 KB in 10 pages; pages guarded by the workgroup barrier	Same as gfx942
Triton host-DSL limits	Primitives complete: atomics, nanosleep via inline asm	AMD backend has buffer_atomic_rmw and buffer_load_to_local, no s_sleep / setprio hooks; poll backoff via inline asm; gfx942 has no async copy (gfx950 does)	Same as gfx942

Every cell comes from the LLVM gfx942 memory model, the CDNA3 ISA or the ROCm atomics documentation, not from conjecture; for a new architecture this table is the first thing to fill in. The generator does one thing: it looks up the event's scope and assembles the instruction sequence. In code (codegen/lowering.py) the generated etx_lowering.h defines ETX_RELEASE_<SCOPE>, ETX_ACQUIRE_<SCOPE>, ETX_POLL_<SCOPE>, ETX_ARRIVE_<SCOPE>, ETX_BACKOFF and ETX_DOMAIN_ID from the YAML rows; primitives.h builds etx_wait_<SCOPE> (spin on ETX_POLL with ETX_BACKOFF) and etx_arrive_<SCOPE> (returns the count after decrement) from those macros and names no architecture.

10.2 Worker loop

Each worker's loop: fetch a task (static table or pop), wait on its in-events (spin, backoff, acquire), execute the tile body (a DSL-compiled function), arrive on its out-events (release plus atomic decrement), and if the count reached zero in a dynamic segment push the consumers (early push optional). Every iteration checks the abort flag and the step-end condition. Static segments have no push, dynamic segments have no pre-ordered table, hybrid has both. Runtime state is only the event buffer (integers), the queues (local plus global), the descriptor table, the shape scalars and the worker-to-domain table; there is no task-graph object and no interpreter. This continues the paper's minimal runtime: scheduling logic is compiled into the kernel, and the three modes differ only in the "fetch" and "push" boxes.

```
persistent_kernel(events, queues, desc_table, shape, dom_map, ctrl):
    w = worker_id(); d = discover_domain(w)      # discovered at launch, no assumed formula
    sched = init_scheduler(d, mode_of_subgraph)  # static table / local queue / both
    while true:
        task = sched.next()                     # static table or pop; if both empty
    consult ctrl
        if task.none: if ctrl.step_done(): break else: backoff(); continue
        wait_all(task.in_events)                # generated from ETensor dependencies
        etx_tile_dispatch(task.type, ctx(task))  # switch into the DSL tile bodies
        for ev in task.out_events:
            if etx_arrive(ev, ev.scope) == 0 and ev.dynamic: sched.push(consumers_of(ev, task))
        if ctrl.abort: break
```

The emitted kernel (codegen/kernel.py) follows this shape. Thread 0 of each workgroup discovers the domain (HW_ID when wg_to_domain_map is discover, else the host-provided worker_domain table), claims a slot within that domain with an atomic, and forms the logical worker id domain × workers_per_domain + slot, so static queues are domain-affine under any workgroup-to-XCD mapping (the measured (k+6) mod 8 included). Each task type receives its own argument pointer table in the grid's declared order; the first hardware run returned zeros because tiles indexed the global table. The workgroup then loops: take the static head if etx_deps_ready (v0: always true), else try the domain-local ticket queue, else the global ticket queue, else take the static head anyway and spin inside the generated wait code. With nothing available it checks ctrl_abort (host writes 1) and ctrl_done >= n_tasks, backs off, and after spin_limit idle iterations writes 2 into ctrl_abort and exits: the on-device watchdog. Per task type the generated code waits on every in-edge target enumerated from the edge map, applies ETX_ACQUIRE_<scope>, calls the tile body, applies ETX_RELEASE_<scope>, arrives on every out-edge target, and in dynamic or hybrid mode pushes the consumers of any event whose count reached zero; each completed task increments ctrl_done.

10.3 Ticket queues

The v0.1 design used a CAS-based ring. Measured on MI300X (section 8.6), a CAS pop costs about 1.1 us alone and its retry storm collapses aggregate throughput past 8 poppers. The v0.2 runtime uses a many-producer many-consumer ticket ring: etx_push does one atomicAdd on tail and one atomicExch into the slot; etx_try_pop first reads head and tail with a device-scope poll and returns empty without reserving if head >= tail, otherwise takes a ticket with one atomicAdd on head and then polls its own slot until a task id appears. Pass 5 sizes each ring's capacity to at least the number of pushes per step, so pushes never wrap, and the host resets the rings between steps. Push lists encode the destination queue: a non-negative task id means the pusher's domain-local queue (hybrid), a bitwise-complemented id means the device-global queue (dynamic). The host seeds the initially ready dynamic and hybrid tasks into the rings before launch.

The first hardware run exposed a wrap-around defect in the original scheme, under which a ticket taken beyond the final tail was assumed harmless: over-popping workers wrapped onto slots consumed in an earlier pass over the ring and re-executed those tasks; the execution trace showed 267 of 320 tasks executed twice and event counters at -4. The rule now is that every slot value carries a pass tag, `value = (ticket / capacity) << 24 | task id`, and a popper treats a tag mismatch as “not yet pushed” and keeps polling. This is the only change needed because pushes cannot wrap.

10.4 Kernel family: multiple devices and resource classes

One Step may be executed by several kernel instances, each a `(device, resource_class)` pair, all sharing one event tensor (plain or uncached memory on one device, fine-grained across devices). Three cases take the same path:

- MI250X: two GCDs are two devices; one card launches two instances; cross-GCD events are system scope.
- 8-card TP: eight instances; cross-card events are system scope; communication tiles are inlined into each instance.
- Register-union split: two resource classes on one device (heavy GEMM / light GEMV); two instances each own part of the CUs; events are device scope.

The scheduler does not know about instances, only the domain tree; an instance is a subtree of that tree handed to one kernel. The code generator emits one translation unit per instance (`megakernel_d<i>.hip`), and `gfx90a.yaml` declares `devices_per_package: 2` so the pipeline emits two.

10.5 Communication fusion without multimm

The paper’s 1.40x on GEMM + Reduce-Scatter rests on `multimm.ld_reduce` (NVLink SHARP), which CDNA lacks. Three candidate paths, each to be tested separately; if none yields a positive gain the capability table records `comm_fusion_reduce_scatter: unsupported` and only compute-side fusion is done:

1. Owner-side reduction: each rank’s GEMM tile writes its result into the owner rank’s fine-grained buffer; the owner’s RS tile waits on the event and reduces in its local L2. The write is a one-way P2P store; the event is a cross-card arrive.
2. P2P floating-point atomic accumulation: MI300 supports FP atomics over IF, MI200 does not; summation order is non-deterministic, so fp32 accumulation with a single bf16 rounding is required.
3. Borrow rocSHMEM / Iris one-sided communication primitives as tile bodies and use ETX only as the scheduler.

The `gfx942` and `gfx950` tables carry `comm_fusion_reduce_scatter: experimental`; `gfx90a` carries `unsupported`; `sm_90` and `sm_100` carry `supported`.

11. Dynamic Workloads

Level	Examples	ETX mechanism	Paper coverage
Shape dynamism	B and S changing under continuous batching; mixed prefill / decode	Symbolic dimensions plus per-step shape scalars; static segments use shape buckets (rather than “the next larger sampled shape”), and the worker count inside a bucket scales with tile count	Covered

Level	Examples	ETX mechanism	Paper coverage
Data-dependent dynamism	MoE routing; accepted length in speculative decoding; block selection in sparse attention	Runtime-initialised <code>wait_count</code> plus <code>range / gather</code> edges; the domain-local dynamic queue absorbs imbalance	MoE covered; speculative decoding is a new instance of the same mechanism (the accepted length decides the trigger range of subsequent tiles)
Cross-step dynamism	A resident kernel across many decode steps; requests joining and leaving at any time; multi-tenant co-location	Host-device control ring: the host writes step descriptors (shape scalars, pointer table, event initial values) into a ring buffer and the kernel reads the next one when a step's events have all reached zero; an abort flag; events carry an epoch stamp (mKernel's approach) so counters need not be cleared between steps and <code>wait</code> compares <code>count == target(epoch)</code>	Not covered (one launch per step); MPK does in-kernel admission; Blink puts the whole serving loop in the kernel

Cross-step control ring: the host engine scheduler (continuous batching) writes step k , $k+1$, ... into a ring buffer in fine-grained memory; the resident megakernel reads the next descriptor at step end and instantiates that step's coordinate space; the abort / pre-emption flag is written by the host and read by every worker on every iteration. Residency is optional: it buys zero launch overhead and earlier weight prefetch at the cost of a kernel that occupies the GPU with pre-emption only through the flag. Under multi-tenancy, or when the GPU is shared with other kernels, it should be off.

All three levels share one principle: the compiled artefact is a template, never recompiled for a concrete value; everything that varies is a launch parameter or a runtime tensor. In code, `ir/instantiate.py` instantiates the symbolic graph per step and examples supply `bindings()` and `runtime()` for their dynamism; dynamic consumer lists are currently materialised host-side per step.

12. Workload Coverage

Workload	Dynamism	Expected schedule	Main source of gain	Risk
Low-batch decode (dense)	shape	static	Breaking operator boundaries: Q and K norm + RoPE in parallel, pipelining between GEMMs, weight prefetch, hundreds of launches removed	Gain consumed when tile quality trails hand-written libraries; at TP=4 the paper found CPU-side overhead to be the bottleneck
MoE decode / small-batch prefill	data-dependent	hybrid	Intra-domain load balancing + two-stage GroupGEMM pipelining + removal of wave quantisation	Cross-domain push cost; with extremely uneven routing the local queue cannot help either
TP communication fusion (GEMM+RS / AG+GEMM)	communication jitter	RS: dynamic (intra-domain); AG: static	Compute-communication overlap	CDNA has no multimem; needs the section 10.5 alternatives
Large-batch prefill	shape	static or unfused	Limited gain; mainly no regression	Regression-test item: the paper claims no large-batch regression, and this is written as a test

Workload	Dynamism	Expected schedule	Main source of gain	Risk
Speculative decoding	data-dependent (accepted length)	hybrid	Verification and next-step draft tiles trigger by accepted length with no host round trip	Tree-draft dependencies are not affine; restrict to chains or fixed trees
Multi-model / multi-tenant co-location	cross-step	isolation by domain	Domain-level isolation; tenants do not slow each other	Queue and memory isolation; pre-emption only through the flag
Non-LLM tile DAGs (diffusion, vision encoders)	mostly static	static	Generality proof	Reference implementations and baselines are missing

13. Verification and Validation Strategy

Correctness:

- Protocol simulator. The event protocol is simulated on the host in Python over the whole task graph for sampled shapes and random routing, checking no deadlock, count conservation, and that every tile's inputs were written before it executes. fleet-mi300x had a prototype (a 3-token simulation plus 12 mutation tests with injected defects); `etx/sim/protocol.py` runs the deadlock check and reports makespan and the busy / wait / scheduling budget per plan.
- Differential testing. Every fused kernel is paired with an unfused reference (per-operator submission, and the same tile code with global barriers); random shapes, random routing, element-wise comparison. This is the only way to separate fusion gain from operator quality.
- Scope tests. Deliberately constructed cross-XCD and cross-card events verify that they are actually visible; this catches silently downgraded system-scope atomics. The 2026-09-22 flag-latency runs (0 stale payloads in 20,000 trials) are the first instance.
- Atomic capability assertion. One atomic write-and-read-back on the target memory type; if the semantics do not hold, compilation fails.
- Watchdog. Every test has a timeout; a hang is a failure and dumps the event tensor (which event's count did not reach zero, who did not arrive). The generated kernel's `spin_limit` and `ctrl_abort = 2` provide the on-device half; the host side aborts after a timeout and dumps every non-zero event counter plus a per-task execution-count histogram. This dump is what located both hardware defects on 2026-09-22 (267 of 320 tasks executed twice with counters at -4; zeros from stale L2 under global scheduling).

Performance:

- Three baselines reported separately: per-operator submission; the same tile code plus global barriers (isolating "fusion gain"); official libraries (cuBLAS / RCCL, CK / hipBLASLt).
- A fixed shape set per pull request with alerts on threshold; every number carries shape, batch and parallelism.
- Scheduling-decision logs are stored so "why did this subgraph choose static" can be answered afterwards.

Portability regression:

- The same IR runs the same small model on gfx90a / gfx942 / gfx950 / sm_90: element-wise identical, no hang.

- The decision log must print the capability differences, proving the decision read the machine model rather than coinciding.
- Every discover field in the capability table (such as the XCD map) is probed in CI and reconciled with the YAML.

14. Implementation Status and Repository Layout

Repository: /Users/ckl/code/etx. Layout by layer:

Path	Layer	Content
etx/ir	L2	types.py (Scope, InitKind, Tensor, ETensor, Resource, TileBody, TaskGrid, Graph), edgemap.py (four edge-map forms, C emission), dims.py (symbolic dimensions), instantiate.py (per-step instantiation), verify.py (checks 1-4)
etx/machine	L3	model.py (MachineModel, effective scopes, cost accessors) and arch/ with gfx90a.yaml, gfx942.yaml, gfx950.yaml, sm_90.yaml, sm_100.yaml
etx/passes	L4	p1_tiling.py, p2_affinity.py, p3_event_elim.py, p4_schedule_mode.py, p5_queues.py, p6_memory.py, p7_prefetch.py, plan.py (Plan, verify_plan), pipeline.py
etx/codegen	L5	lowering.py (YAML rows to etx_lowering.h), kernel.py (persistent-kernel emitter, plan.json)
etx/runtime/include/etx	L5	abi.h, primitives.h (wait / arrive / ticket queues / discovery built on the macros), primitives_amdgcn.h, primitives_nvptx.h
etx/frontends	L1	tileop.py, linkmode.py, triton_host.py
etx/sim	-	protocol.py: deadlock, makespan, critical-path budget
etx/tools	-	cli.py: the archs, explain and compile [--sim] subcommands of python -m etx
examples/	-	splitk_sum.py (paper figure 3; real tiles in tiles/splitk.hip, host launcher in hosts/splitk.hip), moe_layer.py (both dynamisms), gemm_rs.py (two devices); MoE and GEMM + RS tiles are ABI-correct stubs
bench/calib/	-	atomic_pingpong.hip, flag_latency.hip, queue_contention.hip, run_calib.sh, README (also lists phase_switch, discover_domain, p2p_atomic as planned programs)
tests/	-	test_edgemap.py, test_ir_verify.py, test_machine.py, test_passes.py, test_sim.py, test_codegen.py, test_no_arch_branches.py; 46 tests collected and passing (also on the MI300X VM)
docs/	-	ARCHITECTURE.md (code map), DESIGN-v0.1.md, site/index.html (published specification)

What runs today without a GPU: make setup, make test, python -m etx compile examples/moe_layer.py --arch gfx942 --out build/moe_gfx942 --sim. compile verifies the graph, runs the seven passes, and writes megakernel_d0.hip, the lowering header, tile prototypes, plan.json and decisions.log (why each subgraph is static / dynamic / hybrid; which events were eliminated); --sim runs the protocol simulator.

Known gaps (from ARCHITECTURE.md and README.md):

- etx_deps_ready always returns true (the static head is taken before the queues); the planned refinement is a per-task ready bitmap.

- Dynamic consumer lists are materialised host-side per step; the in-kernel inverse-map alternative is not implemented.
- Sentinel-value signalling is a capability bit and a helper (etx_amdgc_n_load_agent), not yet a lowering option chosen by the cost model.
- Tile bodies in examples/tiles/ are stubs with the right ABI; the Triton host-DSL emitter produces a skeleton, not a runnable kernel.
- Cost tables for gfx90a, gfx950, sm_90 and sm_100 are public seed values; gfx942 has the first measured values (Appendix C) to be written into the YAML.
- Only the split-K example has executed on a GPU (MI300X, 2026-09-22, all three modes, reference-checked; section 8.6). MoE and GEMM + reduce-scatter tiles are still stubs; CUDA emission is untested on hardware; the cross-device cost t_dev_ns is uncalibrated; the Triton emitter is a skeleton only.

Runtime details fixed by the hardware run and now part of the design (sections 9.4, 10.2, 10.3, 13): domain-affine logical worker ids claimed by atomic after HW_ID discovery; a per-type argument pointer table; host seeding of initially ready dynamic and hybrid tasks; pass-tagged ticket-ring slots; DEVICE scope for events of globally scheduled grids; and a host watchdog that dumps non-zero event counters and a per-task execution-count histogram.

15. Roadmap and Milestones

Phase	Deliverable	Completion criterion
0. Baseline and calibration	Per-operator MoE / dense reference on MI300X, timing scaffold, seven synchronisation-cost microbenchmarks	Unfused baseline in hand; t_local / t_cross / t_dev and the contention curve in the YAML. Status: atomic ping-pong, flag latency and queue contention measured on 2026-09-22 (Appendix C); memory-type comparison, P2P, static-queue-length and routing-imbalance experiments remain
1. Single-card closed loop	L2 IR + L5 codegen (link mode, hand-written tiles), static scheduling, events partitioned by XCD	MoE layer at least 1.05x the unfused baseline; differential test passes; protocol simulator in CI. The 1.05x is a self-imposed floor: below it, fusion gain is being eaten by something else, which must be diagnosed before continuing. Status: the closed loop (compile, cooperative launch, reference check, watchdog) is proven on the split-K example in all three modes; MoE tiles are the remaining work
1.5. Critical path	Pass 3 event elimination + Pass 7 cross-barrier prefetch + sentinel-signal lowering; per-phase trace tooling	Global events per MoE layer down to the natural dependency count (3); fixed-latency share per layer from about 75% to under 50%; MBU report in CI
2. Dynamism and tables	Local queues, hybrid scheduling, capability + cost tables, decision log	Hybrid at least static on MoE (reference: the paper's 1.08 vs 1.04); logs explainable
3. Second frontend + second hardware	Triton host-DSL mode; the same IR on gfx950 or gfx90a (two instances)	Switching frontend / hardware changes only YAML and the skeleton template; IR and passes unchanged
4. Communication fusion	GEMM + RS on MI300X without multimem	Report a positive gain if there is one; otherwise mark unsupported in the capability table
5. Cross-step residency	Control ring, abort, shape buckets	TPOT under continuous batching no worse than per-step launch, with a bounded pre-emption latency

The seven phase-0 microbenchmarks: (1) atomic-counter round trip between two workgroups in one XCD (t_{local}); (2) the same for adjacent and farthest XCDs (t_{cross} , two values); (3) event tensor in plain VRAM, uncached and fine-grained memory; (4) two-card P2P atomic + fence latency and bandwidth (t_{dev}); (5) throughput of N workgroups contending for one queue, sweeping N (contention curve); (6) static queue length versus straggler time (the boundary of static scheduling); (7) MoE routing imbalance versus dynamic-scheduling gain (reproducing the conditions of the paper’s 1.08 vs 1.04).

16. Risks and Mitigations

Risk	Why	Test first	Fallback
AMD has no multitem	The paper’s communication fusion depends on it	Single-point experiments on the three section 10.5 paths	Mark unsupported; compute-side fusion only
Cross-XCD synchronisation is expensive	4 MB private L2; only the Infinity Cache across domains	Phase-0 calibration (first values: about 0.92 us one-way flag with payload; counters about 100 ns dearer cross-XCD)	Raise the intra-domain partition share at the expense of parallelism
Cluster / DSMEM capability varies by target	LLVM: unsupported targets degrade to agent scope	Measure per target and record in YAML	Global atomics everywhere
Dynamic-queue contention	The paper acknowledges contention on its central queue	Contention-curve calibration (done: CAS collapses past 8 poppers; ticket ring scales to 512)	Domain-level queues (already the default) and the ticket ring (already the runtime)
Tile quality below hand-written libraries	ETC acknowledges it	Like-for-like comparison	Decouple fusion from tiles; inline hand-written tiles
Static scheduling’s shape fallback is coarse	The paper reuses “the next larger shape”	Shape long-tail experiment	Shape buckets plus intra-bucket dynamic scheduling
Wrong event memory type	System-scope atomics have memory-type requirements	Scope tests	Compile-time assertion forbids illegal combinations
Persistent kernels are hard to debug	A hang raises no error; a wrong scope “completes” with wrong data	Watchdog and event dump before any feature work (done: it found both 2026-09-22 defects)	None; this is a prerequisite

Appendix A. Machine Model Schema

Trimmed excerpt of `etx/machine/arch/gfx942.yaml` with comments. Sources for the entries: ROCm MI300 microarchitecture document, CDNA3 ISA section 9.1 (cache-scope bits), LLVM AMDGPUUsage “Memory Model GFX942”, ROCm GPU atomics guide, Chips and Cheese MI300X microbenchmarks. The costs: values shown are the public seed values that phase-0 calibration overwrites; the measured replacements are in Appendix C.

```

name: gfx942
family: CDNA3
vendor: amd
wave_size: 64
devices_per_package: 1
exec_domains:                                # the tree; one entry per level below the
device
  - {name: xcd, count: 8, cus: 38, l2_mb: 4}    # 40 CUs per XCD, 38 active
shared_cache: {kind: infinity_cache, mb: 256, coherent: false} # memory-side; holds the
snoop filter
resources:
  lds_kb: 64
  regs_per_lane: 512          # VGPR + AGPR share one 512-entry file
  max_waves_per_simd: 8
  simd_per_cu: 4
  max_wg_per_cu: 2
  lds_page_kb: 16
bandwidth: {hbm_tbs: 5.3, cache_bw_tbs: 4.3}
visibility:                # correctness: what each event scope needs (LLVM gfx942
memory model)
  domain:                  # within one XCD (one L2). L1 is write-through; consumer
invalidates L1 only
  release: ['asm volatile("s_waitcnt vmcnt(0)" ::: "memory");']
  acquire: ['asm volatile("buffer_inv sc0" ::: "memory");']
  poll: '__hip_atomic_load({ptr}, __ATOMIC_RELAXED, __HIP_MEMORY_SCOPE_AGENT)'
  arrive: '__hip_atomic_fetch_sub({ptr}, 1, __ATOMIC_RELAXED, __HIP_MEMORY_SCOPE_AGENT)'
  memory: any
  device:                  # across XCDs. Producer writes back dirty L2 lines; consumer
invalidates
  release: ['asm volatile("buffer_wbl2 sc1\n\t s_waitcnt vmcnt(0)" ::: "memory");']
  acquire: ['asm volatile("buffer_inv sc1" ::: "memory");']
  poll: '__hip_atomic_load({ptr}, __ATOMIC_RELAXED, __HIP_MEMORY_SCOPE_AGENT)'
  arrive: '__hip_atomic_fetch_sub({ptr}, 1, __ATOMIC_RELAXED, __HIP_MEMORY_SCOPE_AGENT)'
  memory: any
  system:                  # across devices. Coarse-grained memory silently downgrades
to device scope
  release: ['asm volatile("buffer_wbl2 sc0 sc1\n\t s_waitcnt vmcnt(0)" ::: "memory");']
  acquire: ['asm volatile("buffer_inv sc0 sc1" ::: "memory");']
  poll: '__hip_atomic_load({ptr}, __ATOMIC_RELAXED, __HIP_MEMORY_SCOPE_SYSTEM)'
  arrive: '__hip_atomic_fetch_sub({ptr}, 1, __ATOMIC_RELAXED, __HIP_MEMORY_SCOPE_SYSTEM)'
  memory: fine_grained
capabilities:                # correctness: booleans and enums; wrong = wrong code
  cluster_launch: false
  dsmem: false
  multicast_reduce: false    # no multitem, and no switch: 8 GPUs are a point-to-point
xGMI mesh
  async_copy_to_lds: dword_only # buffer_load ... lds is 32 bit/lane; no TMA / cp.async
  tma: false
  fp_atomic_over_fabric: true
  wg_to_domain_map: discover  # docs say round-robin; not contractual; read HW_ID.XCC_ID
at start
  cooperative_launch: true
  sentinel_signal: true
  hw_tile_trigger: false
  backoff: s_sleep
  comm_fusion_reduce_scatter: experimental # no multitem path; see section 10.5
costs:                        # performance: numbers; wrong = slow. Seed values below,
overwritten by bench/calib
  t_local_ns: 116             # same-XCD atomic round trip (Chips and Cheese)
  t_cross_ns: 203             # cross-XCD atomic round trip
  t_flag_cross_ns: 703        # cross-XCD one-way flag latency

```

```

t_fence_ns: 115                # agent-scope release fence
t_phase_switch_counter_ns: 7600
t_phase_switch_sentinel_ns: 850
t_dev_ns: null                 # xGMI P2P atomic, to calibrate
t_pop_ns: 300
t_push_ns: 300
t_prologue_ns: null
lowering:                      # misc C snippets referenced by the generated header
  backoff: '__builtin_amdgcn_s_sleep(1);'
  timer: '__builtin_amdgcn_s_memrealtime()'
  domain_id: 'etx_amdgcn_xcc_id()'          # runtime/include/etx/primitives_amdgcn.h
  launch: hipLaunchCooperativeKernel
notes:
  - "Plain loads hit the non-coherent L1 (wave scope = Hit LRU); flags must be polled with
    sc1 or atomic loads."

```

For contrast, `sm_90.yaml` declares one domain (`{name: gpu, count: 1, cus: 132, l2_mb: 50}`), a coherent shared cache, `lds_kb: 228`, `regs_per_lane: 255`, `wave_size: 32`, only device and system visibility rows (`__threadfence()` / `__threadfence_system()` with acquire loads), capabilities `cluster_launch`, `dsmem`, `multicast_reduce`, `tma`, `pdl` all true, `wg_to_domain_map: single`, `backoff: nanosleep`, and costs `t_local_ns: 600`, `t_dev_ns: 2000`, `t_dsmem_ns: 110`, `t_pop_ns: 250`. The H100's two L2 partitions (far hit 414 vs near 258 cycles) are treated as a cost, not a visibility difference.

Appendix B. Runtime ABI

From `etx/runtime/include/etx/abi.h`, shared by generated kernels, tile bodies and the host launcher.

```

typedef int32_t etx_event;      // one counter; an ETensor is a contiguous run of
them

struct etx_task {
  int32_t type;                 // task-type id == index into the dispatch switch
  int32_t coord[4];
};

struct etx_queue {              // ring of task ids; many-producer many-consumer via
atomics
  int32_t* slots;
  int32_t capacity;
  int32_t* head;                // pop cursor
  int32_t* tail;               // push cursor
};

struct etx_ctx {
  int32_t coord[4];
  const int32_t* shape;         // symbolic scalars of this step, in graph.symbols
  order
  void* const* args;           // tensor pointers, indexed by plan.json "args"
  etx_event* events;           // event buffer base
  const int32_t* ev_offset;     // per-event-tensor offset into events
  const int32_t* ev_shape;     // per-event-tensor shape, 4 ints each
  uint32_t domain;
  uint32_t worker;
  void* lds;
};

```

`etx_params` fields, as passed to `etx_megakernel`:

Field	Meaning
shape, args, events, ev_offset, ev_shape	As in etx_ctx; copied into every task's context
descs	Task descriptor table (etx_task), indexed by task id
static_queue, static_begin, static_end	Concatenated per-worker static queues and each worker's [begin, end) range
local_queue	One ticket ring per domain
global_queue	Device-global ticket ring; capacity 0 when unused
worker_domain	Optional host-provided worker-to-domain map, used when discovery is not needed
push_index, push_offsets, push_lists	CSR consumer lists: per event id a base offset, then per event coordinate a [b, e) range of encoded consumer task ids
ctrl_abort	Host writes 1 to abort; the kernel writes 2 on spin timeout
ctrl_done	Tasks completed this step; the step ends when it reaches n_tasks
n_tasks, spin_limit, n_events	Step size, watchdog threshold, event count

Since the first hardware run, args in etx_ctx is the per-task-type pointer table in the grid's declared argument order rather than the global table, and the ring slots carry the pass tag described in section 10.3.

Primitives, all built from the generated lowering macros and naming no architecture: etx_wait_DOMAIN / DEVICE / SYSTEM, etx_arrive_DOMAIN / DEVICE / SYSTEM (returns the remaining count), etx_push, etx_try_pop (ticket ring; returns -1 when nothing is available, keeps the ticket in per-worker state), etx_push_consumers, etx_deps_ready, etx_step_done, etx_discover_domain, ETX_BACKOFF, ETX_DOMAIN_ID. AMD helpers in primitives_amdgcn.h: etx_amdgcn_xcc_id() (HW_ID.XCC_ID via s_getreg_b32, bits [3:0], on gfx940-gfx950), etx_amdgcn_poll_l2() (buffer_inv_sc0 then a volatile load), etx_amdgcn_load_agent() (agent-scope relaxed load for sentinel polling).

Appendix C. Phase-0 Calibration Results

All values measured on 2026-09-22 on a rented Hot Aisle 1x MI300X VM, ROCm 7.2.4, target gfx942, using bench/calib/atomic_pingpong.hip, flag_latency.hip and queue_contention.hip. Status: measured.

Measurement	Condition	Value	Implication
Workgroup-to-XCD mapping	Persistent kernel, HW_ID.XCC_ID read per workgroup	Workgroup k on XCD (k+6) mod 8; first 16: 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5	Not the documented round-robin from 0 and not the (k+4) mod 8 reported publicly; discovery at kernel start is mandatory
One-way arrive latency, DEVICE protocol	Producer buffer_wb12_sc1; consumer agent-scope atomic-load poll; same XCD	873 ns	Reference for the current poll row
Same	Cross XCD	892 ns	Counter cost differs by about 20 ns across XCDs under this protocol
One-way arrive latency, INVL1 protocol	No write-back; consumer buffer_inv_sc0 then plain load; same XCD	642 ns	Correct and about 25% cheaper than the atomic-load poll

Measurement	Condition	Value	Implication
Same	Cross XCD	742 ns	Same-vs-cross difference about 100 ns; every RMW executes at the fabric
Poll with plain or sc0 load, no invalidate	Either XCD	Never observes the arrive (spins forever)	Plain loads are never acceptable for flags
One-way flag latency with payload	s_memrealtime (100 MHz); producer stamps payload, buffer_wbl2 sc1, agent store; consumer agent poll + buffer_inv sc1; same XCD	951 ns, 0 stale payloads / 10,000	Release / acquire sequences of the lowering table are correct
Same	Cross XCD	916 ns, 0 stale payloads / 10,000	Replaces the 703 ns seed for t_flag_cross_ns
DEVICE release fence	buffer_wbl2 sc1 + s_waitcnt	181 ns	Per device-scope arrive
DEVICE acquire fence	buffer_inv sc1	120 ns	Per device-scope wait
DOMAIN release + acquire	s_waitcnt + buffer_inv sc0	44 ns together	A domain-scope edge saves about 257 ns of fences per pair plus the payload write-back
Queue: CAS-based pop, aggregate throughput	65,536 pops of one ring; N = 1 / 8 / 64 / 512	0.89 / 1.85 / 1.10 / 0.41 M pops/s	Collapses past 8 poppers; abandoned
Queue: ticket-based pop, aggregate throughput	N = 1 / 8 / 64 / 128 / 256 / 512	1.15 / 8.48 / 47.6 / 65.0 / 55.2 / 76.6 M pops/s	Adopted as the v0.2 runtime ring; feeds C_queue
Uncontended pop cost	Ticket ring, N = 1	about 870 ns	Replaces the 300 ns seed for t_pop_ns
Per-popper pop latency (derived: N / throughput)	Ticket ring, N = 128 / 256	about 2.0 us / 4.6 us	The linear contention factor in Pass 4 matches at 128 and underestimates at 256
Toolchain	Generated split-K and MoE kernels	Pass hipcc -fgpu-rdc -fsyntax-only for gfx942; 46 tests pass on the VM	MoE tiles remain stubs
End-to-end split-K megakernel, correctness	608 cooperative workers (8 XCDs x 76 workgroups, 2 per CU, 256 threads); static, hybrid and dynamic modes	Matches CPU fp32 reference, max abs error 4.8e-7, in all three modes	First ETX-generated kernel to run; lowering, queues, discovery and per-type argument tables validated
Split-K per-step time, n=64 (320 tasks, 64 events)	Best of 30-50 repetitions; first launch about 21 ms including code load	static 0.101 ms, hybrid 0.115 ms, dynamic 0.118 ms	Queue overhead small relative to the step at this size
Split-K per-step time, n=1024 (5120 tasks, 1024 events)	Same	static 0.128 ms, hybrid 0.441 ms, dynamic 0.512 ms	Tasks about 2 us; about 1 us per pop plus a push; static wins 3.5-4x on tiny tasks, as the C_queue term predicts

Measurement	Condition	Value	Implication
Ticket-ring wrap-around defect	Over-popping workers wrapped onto slots consumed in an earlier pass	267 of 320 tasks executed twice; event counters at -4	Slot values now carry a pass tag $((\text{ticket} / \text{capacity}) \ll 24)$ OR-ed with the task id; a tag mismatch means “not yet pushed”
Scope under global scheduling defect	Grid scheduled through the device-global queue with DOMAIN-scope events	Consumers read stale L2 and produced zeros while the kernel “completed”	Pass 4 raises events of dynamically scheduled grids to DEVICE scope; <code>verify_plan</code> rejects violations

Appendix D. Glossary

Term	Meaning
task / tile	One workgroup-level unit of work: a tile body at one coordinate of a task grid
task grid	A tile body launched over a symbolic multidimensional coordinate space
event	One integer counter; producers arrive (decrement), consumers wait (spin until zero)
event tensor (ETensor)	A multidimensional array of events; a first-class IR object lowered to one integer tensor
edge map	Map from task coordinates to event coordinates: affine, index arithmetic, runtime range, runtime gather
scope	Visibility range of an event: workgroup < cluster < domain < device < system
exec domain	One level of the execution-resource tree: XCD, GCD, die or whole GPU
capability / cost	A hardware bit or enumeration deciding correctness / a coefficient deciding performance
worker	One persistent workgroup executing the worker loop
static / dynamic / hybrid	Pre-ordered per-worker queues with arrive / wait only; ready queues with push / pop; dynamic inside a domain and static across domains with no cross-domain push
resource class / kernel family	Tiles with compatible register / LDS footprint; the set of kernel instances, one per (device, resource class), sharing one event tensor
early push	Pushing a consumer when its producer is dispatched rather than when the event fires (ETC appendix E)
epoch event / control ring	An event compared against a per-step epoch target; the host-to-kernel ring of step descriptors
MBU / TPOT	Model bandwidth utilisation (bytes per token / (HBM bandwidth x step time)); time per output token

Appendix E. References

- Jin, Hou, Wang, Lai et al. Event Tensor: A Unified Abstraction for Compiling Dynamic Megakernel. MLSys 2026. <https://arxiv.org/abs/2604.13327> (all “paper” numbers are from v2)
- Cheng et al. Mirage Persistent Kernel: A Compiler and Runtime for Mega-Kernelizing Tensor Programs. <https://arxiv.org/abs/2512.22219>
- Spector et al. Look Ma, No Bubbles! Designing a Low-Latency Megakernel for Llama-1B. <https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles>

- Zheng et al. Triton-Distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. <https://arxiv.org/abs/2504.19442>
- Hou et al. Axe: A Simple Unified Layout Abstraction for Machine Learning Compilers (ETC's tile DSL). <https://arxiv.org/abs/2601.19092>
- AMD. Fleet: chiplet-aware megakernel on MI350X (MPK's ROCm port; four-level tasks, hierarchical events, one scheduling workgroup per XCD). <https://arxiv.org/abs/2604.15379> ; <https://github.com/ROCm/fleet-chiplet-megakernel>
- Cohere. Megakernels (2026-09; H100; static waves plus per-operator drain queues; 30B-A3B bs=1, 1.58x over vLLM). <https://cohere.com/blog/megakernels>
- Kog. Building a single-kernel latency-optimized LLM inference engine on AMD MI300X GPUs (sentinel polling 0.8 us vs counters 7.6 us). <https://blog.kog.ai/building-a-single-kernel-latency-optimized-llm-inference-engine-on-amd-mi300x-gpus/>
- Ada-MK. <https://arxiv.org/abs/2605.11581>
- ForgeMegakernel. <https://arxiv.org/abs/2609.12379>
- mKernel. <https://arxiv.org/abs/2609.13585>
- DITRON. <https://arxiv.org/abs/2605.02953>
- Iris. <https://arxiv.org/abs/2511.12500> ; <https://github.com/ROCm/iris>
- Chips and Cheese. Testing AMD's giant MI300X (same-XCD / cross-XCD atomics 116 / 202 ns). <https://chipsandcheese.com/p/testing-amds-giant-mi300x>
- Hopper microbenchmarks (DSMEM and L2 partition latency). <https://arxiv.org/pdf/2501.12084>
- NVIDIA. Blackwell tuning guide. <https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html>
- NVIDIA. PTX ISA (multimem, nanosleep, griddepcontrol). <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html>
- AMD. MI300 microarchitecture. <https://rocm.docs.amd.com/en/latest/reference/gpu-arch/mi300.html>
- AMD. MI350 microarchitecture. <https://rocm.docs.amd.com/en/latest/reference/gpu-arch/mi350.html>
- AMD. SPX / CPX partitioning and round-robin dispatch. <https://rocm.blogs.amd.com/software-tools-optimization/compute-memory-modes/README.html>
- AMD. device-libs cg.cl (grid.sync implementation). <https://github.com/ROCm/llvm-project/blob/amd-staging/amd/device-libs/ockl/src/cg.cl>
- AMD. ROCm GPU architecture specs. <https://rocm.docs.amd.com/en/latest/reference/gpu-arch-specs.html>
- AMD. GPU atomics operation support. <https://rocm.docs.amd.com/en/latest/reference/gpu-atomics-operation.html>
- AMD. MI300 / MI350 workload optimization (XCD-aware swizzle). <https://rocm.docs.amd.com/projects/ai-ecosystem/en/latest/optimization/workload-optimization.html>
- AMD. MI300X partitioning. <https://instinct.docs.amd.com/projects/amdgpu-docs/en/latest/gpu-partitioning/mi300x/overview.html>
- AMD. AMD Instinct MI300 CDNA3 ISA Reference Guide (2025-08).
- LLVM. AMDGPU backend usage (gfx942 memory model: wbl2 / inv sequences, cluster launch). <https://llvm.org/docs/AMDGPUUsage.html#memory-model-gfx942>
- LLVM. Clang AMDGPU builtins (s_sleep, s_setprio, s_memrealtime). <https://clang.llvm.org/docs/AMDGPUBuiltinReference.html>
- MI250X kernel-initiated P2P latency (8.7-18.2 us). <https://arxiv.org/abs/2410.00801>
- cklxx. fleet-mi300x: hand-written batch-1 MoE decode megakernel on MI300X; cross-XCD event, fence and discovery experience cited from docs/STATUS.md. <https://github.com/cklxx/fleet-mi300x>

- ETX design page (source of this document). <https://etc-dynamic-megakernel-arch.q1293822641.workers.dev>